Full Stack Open

Python Edition

Modern web development with Reflex, FastAPI, SQLAlchemy and PostgreSQL

Chapters 1–4b

Based on the structure of the University of Helsinki's *Full Stack Open* course,
rewritten for a Python technology stack.

Contents

Before you start **iii**

What this book is . . . iii

How the pieces fit together . . . iii

Libraries, frameworks and tools . . . iv

Setting up your machine . . . v

1 Introduction to Reflex **1**

a. Introduction to Reflex . . . 1

b. Python for this course . . . 4

c. Component state, event handlers . . . 10

d. A more complex state, debugging Reflex apps . . . 13

2 Communicating with a server **17**

a. Rendering a collection, modules . . . 17

b. Forms . . . 19

c. Getting data from a server . . . 22

d. Altering data on the server . . . 27

e. Adding styles to a Reflex app . . . 30

3 Programming a server with FastAPI **35**

a. FastAPI basics . . . 35

b. Deploying the app to the internet . . . 43

c. Saving data to PostgreSQL . . . 46

d. Validation and Ruff . . . 61

4 Testing FastAPI apps and user administration **70**

a. Project structure and intro to testing . . . 70

b. Testing the backend . . . 81

Before you start

What this book is

Full Stack Open is a free course from the University of Helsinki that teaches modern web development with JavaScript: React in the browser, Node.js and Express on the server, MongoDB as the database. This book follows the same path, chapter by chapter and exercise by exercise, but every part of the stack is Python:

Job	Original course	This book
Frontend (what the user sees)	React	Reflex
Talking to servers	axios	httpx
Backend (the server)	Node.js + Express	FastAPI + uvicorn
Validating data	Mongoose schemas	Pydantic
Database	MongoDB (Atlas)	PostgreSQL (Neon) + SQLAlchemy
Hosting	Render / Fly.io	Render
Linting	ESLint	Ruff
Testing	node:test + supertest	pytest + FastAPI's `TestClient`

A **full stack** developer works on all of these layers. Each chapter is split into lettered sections (1a, 1b, ...). Every section explains something new and ends with exercises. Exercises marked with a star (*) are harder; you can skip them and still continue.

How the pieces fit together

By the end of chapter 3 your apps look like this:

```
Browser ──WebSocket──> Reflex backend ──HTTP + JSON──> FastAPI backend ──SQL──>
↪  PostgreSQL
(the page)            (your state and handlers)        (your routes)           (the
↪  data)
└──────────── the Reflex app ────────────┘             └─ the backend ─┘
```

- The **browser** shows the page and reports clicks and typing.
- The **Reflex backend** runs your Python state and event handlers. It is part of the frontend app: Reflex apps always have a small server of their own.

- The **FastAPI backend** is a separate program. It receives HTTP requests, checks them, and reads or writes the database.

- **PostgreSQL** stores the data permanently.

Each part has one job and can be replaced on its own. In chapter 3 you will switch the frontend from a fake server to your own FastAPI server by changing one URL, and later switch the backend from a Python list to PostgreSQL without touching the frontend at all.

Libraries, frameworks and tools

Almost nothing in this book is written from zero. Most of the work is done by code other people wrote and published, which you install and use. It helps to know the words for it.

- A **library** is code you *call*. You decide when to use it. Example: `httpx.get(url)` sends one HTTP request when your code asks for it, and then your code continues.

- A **framework** is code that *calls you*. It owns the main loop of the program, and you fill in pieces it calls at the right moment. Example: in FastAPI you write a function for `GET /api/notes`, but you never call it yourself; FastAPI calls it every time such a request arrives. This is sometimes called "don't call us, we'll call you".

- A **tool** is a program you run while developing, not part of the app itself: Ruff checks your code, pytest runs your tests, Git records your history.

- A **service** is something run by a company on its own computers, which you use over the internet: Render runs your apps, Neon runs your database, GitHub stores your repository.

Every one of them exists because doing the job by hand is long, repetitive and easy to get wrong. The table below says what problem each one solves. You don't need to understand it all now; each is explained from scratch in the chapter where it first appears.

Name	Kind	The problem it solves
Reflex	framework	Building an interactive web page normally needs JavaScript. Reflex lets you write the page and its behaviour in Python, and produces the JavaScript for you (chapter 1).
httpx	library	Sending an HTTP request means opening a network connection and writing the request text in the exact format HTTP expects. httpx does that, and turns the reply into Python data (chapter 2).
FastAPI	framework	A server must read raw requests, decide which code answers which URL, check the incoming data and write correct responses. FastAPI does all of that; you only write what each URL should do (chapter 3).
uvicorn	program	Something has to listen on a network port and pass every incoming request to your FastAPI code. That's uvicorn (chapter 3).
Pydantic	library	Data from outside (a request body, a JSON reply) can be missing fields or have wrong types. Pydantic checks data against a description you write as a Python class (chapter 3).
PostgreSQL	program	Keeping data safe on disk, shared by many users at once, without corrupting it. A database server does this (chapter 3c).
SQLAlchemy	library	Talking to a database means writing SQL text and converting the rows that come back into Python objects. SQLAlchemy writes the SQL from Python code and does the converting (chapter 3c).
psycopg	library	The low-level part that actually opens the network connection to PostgreSQL and speaks its protocol. SQLAlchemy uses it underneath (chapter 3c).
python-dotenv	library	Loading secret settings from a file instead of typing them into every terminal (chapter 3c).
pytest	tool	Running your test functions, and reporting clearly which ones failed and why (chapter 4).
Ruff	tool	Finding likely mistakes without running the code, and keeping the formatting consistent (chapter 3d).
Docker	tool	Running programs such as PostgreSQL in an isolated box without installing them on your system (chapter 4b).
Git, GitHub	tool, service	Recording the history of your code, and storing it online (this chapter).
Render, Neon	services	Running your apps and your database on the internet (chapter 3).

Setting up your machine

The terminal

Most work in this book starts by typing a command in a **terminal**. A terminal is a window with a text prompt, and the program behind it that reads your commands is the **shell** (on Ubuntu usually *bash*). You type one line, press Enter, the shell runs it, prints the result, and shows the prompt again.

The working directory. The shell is always "in" one folder, its **current working directory**. Commands that take file names look for them there. Many prompts show it, for example `~/dev/fso-py$`, where ~ means your home folder.

```
pwd                 print the working directory
ls                  list the files in it
ls -a               also list hidden files (names starting with a dot, like .env)
cd chapter3         go into the folder chapter3
cd ..               go up one folder (.. always means "the folder above")
cd                  go back to your home folder
mkdir notes         create a folder
```

A path that starts with `/` is **absolute** (it starts from the top of the disk). Any other path is **relative** to the working directory. That's why the book often says "from the repo root" or "from the `notes_backend` folder": the same command can mean different files depending on where you run it.

Running a program. A command is a program name followed by **arguments**, separated by spaces:

```
uvicorn main:app --reload --port 3001
```

Here the program is `uvicorn`, and it receives three arguments: `main:app`, `--reload`, and `--port 3001`. Arguments that start with dashes are **options** (or flags) that change how the program behaves; some take a value after them. Every program decides for itself which arguments it understands; `program --help` usually lists them. An argument that contains spaces must be quoted, `"Noor Haddad"`, otherwise the shell splits it into two.

A program that keeps running, like a server, keeps the terminal busy. **Ctrl+C** stops it. Open a second terminal (or a second tab) to run another command at the same time; you'll often have a backend in one terminal and a frontend in another.

Environment variables. Every running program receives a set of name–value pairs from the shell that started it, called **environment variables**. They're used for settings that shouldn't be written into the code, such as a password or the address of a server. In bash:

```
export GREETING=hello     set a variable for this terminal and every program started
↪  from it
echo $GREETING            the shell replaces $GREETING with its value before running
↪  echo: prints hello
```

The `$NAME` replacement happens in the shell, before the program even starts. So in `uvicorn --port $PORT`, uvicorn never sees the text `$PORT`; it receives the number. Variables set with `export` disappear when the terminal closes. Chapter 2e shows how Python reads them, and chapter 3c how to keep them in a file.

How the shell finds programs: PATH. When you type `uvicorn`, the shell looks for a program file with that name in a list of folders stored in the environment variable `PATH`, in order, and runs the first one it finds. `which uvicorn` shows which file that is. This is exactly what activating a virtual environment changes, as the next section explains.

Python and virtual environments

You need Python 3.13. Check it with `python3 --version`.

Python packages (Reflex, FastAPI, ...) are installed with **pip**. If you install them globally, every project on your computer shares one set of versions, and upgrading a package for one project can break another. A **virtual environment** (venv) solves this: it is a folder that contains its own Python interpreter and its own packages.

Create one at the root of your course repository and *activate* it:

```
python3 -m venv .venv
source .venv/bin/activate
```

- `python3 -m venv .venv` runs Python's built-in `venv` module, which creates the folder `.venv`.
- `source .venv/bin/activate` puts the folder `.venv/bin` at the *front* of your terminal's `PATH`. From then on, when you type `python`, `pip`, or any program a package installed (`uvicorn`, `reflex`, `pytest`), the shell finds the copy inside `.venv` first. Your prompt shows `(.venv)` while it is active. Every new terminal needs this line again.
- `deactivate` switches back.

From now on, `pip install something` puts packages inside the `.venv` folder. The folder is never committed to Git (it is in `.gitignore`): anyone can recreate it from a list of package versions.

requirements.txt

That list is `requirements.txt`: one package per line, with an exact version.

```
pip freeze > requirements.txt      # write every installed package and its version
pip install -r requirements.txt    # install everything listed in the file
```

Exact versions (`fastapi==0.142.2`) matter: when your app is deployed, the server installs exactly what you tested with. Each app in this book gets its own short `requirements.txt` with only the packages it needs, because the hosting service installs that file.

Git: recording the history of your code

The problem. You change ten files, and the app stops working. Which change broke it? What did the code look like yesterday, when it worked? Copying the project folder by hand every day doesn't scale. A **version control system** records the history of a project for you, as a series of snapshots, and lets you compare any two of them or go back to one. **Git** is the one almost everyone uses.

A repository is a project folder whose history Git tracks. The history itself lives in a hidden folder named `.git` inside it; everything else in the folder is your **working tree**, the files as they are right now.

A commit is one snapshot of the project, plus a message saying what changed, the author and the time. Each commit points to the one before it, so the commits form the history. A commit is identified by a long hexadecimal code, its **hash**, usually shown shortened (`fae5878`).

The staging area. Git doesn't put every change into the next commit automatically. You first choose which changes belong in it by *staging* them. So there are three places a change can be:

```
working tree  ──git add──>  staging area  ──git commit──>  history
(your files)                (the next commit)              (all commits, kept forever)
```

Choosing lets you make commits that each do one thing, even if you changed several unrelated things at once.

```
git init                      turn the current folder into a repository (once)
git status                    which files changed, and which changes are staged
git diff                      the exact lines you changed but haven't staged yet
git add main.py               stage the changes in one file
git add .                     stage every change in the current folder and below
git commit -m "add notes API" create a commit from the staged changes, with a message
git log --oneline             list the commits, newest first
```

Write commit messages that say what changed ("add PATCH route for persons"), and commit often: a small commit is easy to understand, and easy to undo.

.gitignore is a text file listing names and patterns Git should never track: generated folders (`.venv/`, `.web/`, `__pycache__/`) and files with secrets (`.env`). Ignored files don't appear in `git status` and can't be staged by accident with `git add .`.

GitHub, remotes and push. Your repository lives on your computer. **GitHub** is a service that stores a copy of it online: a **remote** repository. After you create an empty repository on GitHub and connect yours to it once (GitHub shows the exact `git remote add origin ...` command), you send your new commits there with:

```
git push
```

Pushing copies your commits to GitHub; it never sends uncommitted changes. Hosting services such as Render (chapter 3) watch your GitHub repository and deploy a new version of your app on every push.

Secrets stay in the history

Every commit is a full snapshot that Git keeps forever. If you commit a file containing a password and delete it in the next commit, the *new* snapshot no longer contains it, but the *old* one still does, and anyone with the repository can open that old commit. Once pushed to a public GitHub repository, consider the password public. That's why secrets go in files listed in `.gitignore` from the start.

The repository layout

Keep all your work in one repository, with one folder per chapter:

```
fso-py/
├── .git/              ← Git's history (hidden)
├── .venv/             ← the virtual environment (ignored by Git)
├── .gitignore
├── README.md
├── requirements.txt   ← everything installed in .venv
├── tools/             ← small helper programs (chapter 2)
├── chapter1/
├── chapter2/
├── chapter3/
└── chapter4/
```

The editor

The book uses VS Code with the Python extension (which includes **Pylance**, a tool that checks your types while you write). Pylance's warnings are worth reading: several sections of this book start from one.

Python uses indentation to decide which lines belong to a block. Tabs and spaces are different characters, and a file must not mix them in a way that could be read differently (chapter 4a explains exactly why). The code in this book is indented with tabs. To make VS Code always insert tabs in Python files, open `.vscode/settings.json` and add:

```
{
    "[python]": {
        "editor.insertSpaces": false,
        "editor.detectIndentation": false
    }
}
```

- `insertSpaces: false`: the Tab key inserts a tab character.
- `detectIndentation: false`: VS Code no longer guesses the style from what a file already contains. Without this, a new or almost empty file can silently switch to spaces.

The bottom-right corner of VS Code shows the current file's mode: "Tab Size: 4" means tabs, "Spaces: 4" means spaces.

Chapter 1

Introduction to Reflex

Sections: a. Introduction to Reflex · b. Python for this course · c. Component state, event handlers · d. A more complex state, debugging Reflex apps

a. Introduction to Reflex

What a web page is made of

When you open a web page, your browser downloads and runs three kinds of files:

- **HTML** describes the page's *structure*: headings, paragraphs, buttons, lists. It is a tree of **elements**, written as tags: `<h1>Notes</h1>`, `<button>save</button>`. An element can contain text and other elements (its **children**), and it can have **attributes**, such as `<img src="flag.png">`.
- **CSS** describes how elements *look*: colours, sizes, spacing (chapter 2e).
- **JavaScript** is the only programming language a browser runs. It makes the page interactive: it reacts to clicks and changes the page without reloading it.

The browser keeps the page as a tree of objects called the **DOM** (Document Object Model). JavaScript changes what you see by changing the DOM.

What React is, and what Reflex adds

The original course writes the frontend in **React**, a JavaScript library. In React you describe the page as **components**: functions that return a piece of the page. When data changes, React calls the components again and updates only the parts of the DOM that are different.

Reflex lets you write the same kind of app entirely in Python. It works in two halves:

1. **Compiling.** Your component functions are ordinary Python functions. When you start the app, Reflex calls them once, collects the tree of components they return, and *compiles* it into a React app (JavaScript), which it saves in the hidden folder `.web/`. That JavaScript is what runs in the browser.
2. **The Reflex backend.** Your *state* (the data that changes) and your *event handlers* (the code that runs when the user clicks) stay in Python and run on a small server that Reflex starts

for you. The browser and that server stay connected through a **WebSocket**, a network connection that stays open so messages can go both ways at any time.

So a Reflex app is really two programs: JavaScript in the browser that draws the page, and Python on a server that holds the state. Section 1c follows one click through both of them.

Creating an app

Install Reflex in your venv and create the first app:

```
pip install reflex
mkdir -p chapter1/intro && cd chapter1/intro
reflex init
```

Choose the blank template. `reflex init` creates:

```
intro/
├── rxconfig.py        ← app settings (the app's name, plugins)
├── requirements.txt   ← the Python packages this app needs
├── assets/            ← static files: images, the favicon, CSS (chapter 2e)
├── .gitignore         ← already ignores .web/ and other generated files
└── intro/
    ├── __init__.py
    └── intro.py       ← your code
```

The inner folder has the same name as the app: Reflex imports `intro/intro.py` and looks for a variable named `app` in it. Start the app with:

```
reflex run
```

The first start is slow because Reflex installs the JavaScript packages it needs. Open `http://localhost:3000`. `localhost` is a name that always means "this computer". The number after the colon is a **port**. One computer can run many programs that accept network connections, so each program listens on its own port number, and a connection names the port to say which program it's for, like an apartment number in a street address. Two servers are now running:

- port **3000**: the **frontend** server, which sends the compiled JavaScript to the browser;
- port **8000**: the **Reflex backend**, which runs your state and handlers.

While `reflex run` is running, it watches your files: save a change and the page reloads by itself. Stop it with Ctrl+C.

Components

Replace the contents of `intro/intro.py`:

```python
import reflex as rx


def index() -> rx.Component:
    return rx.el.div(
        rx.el.p("Hello world"),
```

```
    )


app = rx.App()
app.add_page(index)
```

- `import reflex as rx` imports the library under the short name `rx`.
- `index` is a **component**: a function that returns an `rx.Component`. `-> rx.Component` is a *type hint* that says what the function returns (section 1b).
- `rx.el.div(...)` and `rx.el.p(...)` create the HTML elements `<div>` and `<p>`. Everything under `rx.el` is a plain HTML element with the same name. (Reflex also has ready-styled components such as `rx.button`; this book uses the plain `rx.el` ones, so what you write maps directly to HTML.)
- `app = rx.App()` creates the app, and `app.add_page(index)` makes `index` the page shown at `/`.

Children and props. The *positional* arguments of a component are its children: text or other components. The *keyword* arguments are its **props** (properties), which become HTML attributes or event handlers:

```
rx.el.img(src="https://example.com/flag.png")      # prop: src
rx.el.button("save", type="submit")               # child: "save", prop: type
```

Because components are plain functions, you can compute values with normal Python before returning:

```
def hello(name: str, age: int) -> rx.Component:
    return rx.el.p(f"Hello {name}, you are {age} years old")


def index() -> rx.Component:
    now = 2026
    return rx.el.div(
        rx.el.h1("Greetings"),
        hello("Maren", 2026 - 1999),
        hello("Idris", 26),
    )
```

`hello` is a component of your own. It receives data through its **parameters**, the same role props play in React. Splitting a page into small components, each with one job, is how every app in this book is built.

Fragments. A component returns exactly one root component. When you want to return several siblings without wrapping them in an extra `<div>`, use `rx.fragment(...)`: it groups its children but adds nothing to the HTML.

Remember: these functions run once

`index()` is called when Reflex *compiles* the app, not every time the page is shown. A line like `now = datetime.now()` inside a component would freeze the time of compilation into the page. Data that changes while the app runs belongs in the *state*, which section 1c introduces.

Exercises 1.1–1.2

Create each exercise app as its own Reflex app inside `chapter1/`. Commit after each exercise.

1.1: Course information, step 1

Create `chapter1/courseinfo` with `reflex init`. Start from this single component:

```
def index() -> rx.Component:
    course = "Half Stack application development"
    part1 = "Fundamentals of Reflex"
    exercises1 = 10
    part2 = "Using arguments to pass data"
    exercises2 = 7
    part3 = "State of a component"
    exercises3 = 14

    return rx.el.div(
        rx.el.h1(course),
        rx.el.p(f"{part1} {exercises1}"),
        rx.el.p(f"{part2} {exercises2}"),
        rx.el.p(f"{part3} {exercises3}"),
        rx.el.p(f"Number of exercises {exercises1 + exercises2 + exercises3}"),
    )
```

Split it into three components: `header` (the course name), `content` (the parts and their exercise counts) and `total` (the number of exercises). Pass all data through parameters; `index` keeps the data and only calls the three components.

1.2: Course information, step 2

Refactor `content` so that it uses a fourth component, `part`, which renders the name and exercise count of one part. `content` calls `part` three times.

b. Python for this course

The original course spends this section on JavaScript. This one covers the Python the rest of the book relies on. If something here is familiar, skim it, but read the parts about type hints, unpacking, comprehensions, lambdas and decorators: they appear on almost every page.

Values, names and types

A Python variable is a *name* that points to a value. Every value has a type (`int`, `float`, `str`, `bool`, `None`, ...). The name itself has no type; the same name can later point to a value of another type.

Type hints write down the type you intend:

```
count: int = 0
names: list[str] = ["Maren", "Idris"]

def total(a: int, b: int) -> int:
    return a + b
```

Python itself ignores hints when running the code: `total("a", "b")` runs and returns `"ab"`. They are read by *tools*: Pylance warns you while you write, and libraries can read them on purpose. Reflex uses them to know the type of each state variable, and in chapter 3 FastAPI uses them to check every request. So in this book, hints are not decoration: they change what the libraries do.

`int | None` means "an `int` or `None`". You will use it for values that may be missing.

Strings and f-strings

```
name = "Maren"
f"Hello {name}, 2 + 3 = {2 + 3}"     # 'Hello Maren, 2 + 3 = 5'
```

An **f-string** (the `f` before the quote) evaluates every `{...}` and inserts the result. Inside the braces you can write any expression. A format spec after a colon controls how a number is printed: `f"{3.14159:.1f}"` gives `'3.1'`.

Lists, dicts and tuples

```
parts = ["Fundamentals", "State"]          # list: ordered, can change
parts.append("Forms")                      # add at the end
parts[0]                                   # 'Fundamentals' (first item)
parts[-1]                                  # 'Forms' (last item)
len(parts)                                 # 3

part = {"name": "State", "exercises": 14}  # dict: key -> value
part["name"]                               # 'State'
part.get("missing", 0)                     # 0: a default instead of an error
part["exercises"] = 15                     # change a value

point = (3, 4)                             # tuple: like a list, but cannot change
x, y = point                               # unpacking: x = 3, y = 4
```

Reading a missing key with `part["missing"]` raises `KeyError`; `.get(key, default)` returns the default instead.

Names point to objects; they don't copy them. After `b = a`, both names point to *the same* list, so `b.append(1)` also changes what you see through `a`. This matters in section 1d.

Functions

```
def greet(name: str, greeting: str = "Hello") -> str:
    return f"{greeting} {name}"

greet("Maren")                    # 'Hello Maren'
greet("Maren", greeting="Hi")     # keyword argument: 'Hi Maren'
```

- `greeting: str = "Hello"` is a **default value**: the argument may be left out.
- Arguments can be passed by position or by name (**keyword arguments**). Reflex and FastAPI use keyword arguments everywhere: `rx.el.button("save", on_click=...)`.

- Functions are values too. You can store a function in a variable or pass it to another function without calling it: `button(on_click=State.click_left)` passes the handler itself, and Reflex calls it later.

Unpacking with * and **

```
numbers = [1, 2, 3]
print(*numbers)            # same as print(1, 2, 3)

data = {"name": "Maren", "greeting": "Hi"}
greet(**data)              # same as greet(name="Maren", greeting="Hi")
```

- `*list` spreads a list into separate *positional* arguments.
- `**dict` spreads a dict into separate *keyword* arguments, one per key.

You will use `*` to pass a list of components as children (`rx.fragment(*[part(p) for p in parts])`), and ** to build objects from JSON in chapter 2 (`Note(**data)`).

Comprehensions

A **list comprehension** builds a new list from another one in one expression:

```
numbers = [1, 2, 3, 4]
[n * 10 for n in numbers]              # [10, 20, 30, 40]
[n for n in numbers if n % 2 == 0]     # [2, 4]: only items that pass the test
```

Read it as: "`n * 10`, for every `n` in `numbers`". It replaces the JavaScript methods `map` (transform every item) and `filter` (keep some items). Without the square brackets, `(n * 10 for n in numbers)` is a **generator expression**: it produces the values one at a time instead of building a list, which is enough when you pass them straight to a function like `sum(...)`.

Lambdas

```
double = lambda n: n * 2
double(4)                                  # 8
sorted(["bb", "a", "ccc"], key=lambda s: len(s))   # ['a', 'bb', 'ccc']
```

A **lambda** is a small function without a name, made of a single expression. `lambda n: n * 2` means "given `n`, return `n * 2`". It is handy when a function is needed only in one place, usually as an argument to another function. Reflex uses lambdas to pass arguments to event handlers (chapter 2).

Classes, objects and self

```
class Counter:
    def __init__(self, start: int):
        self.value = start
```

```
    def increase(self) -> None:
        self.value += 1


c = Counter(5)       # create an object: Python calls __init__(c, 5)
c.increase()         # Python calls Counter.increase(c)
c.value              # 6
```

A **class** is a blueprint, and every **object** (instance) made from it has its own data. `self` is the object a method was called on: `c.increase()` becomes `Counter.increase(c)`, so inside the method `self` is `c`. A class can **inherit** from another class, `class NoteState(rx.State):`, which gives it everything the parent class has. Libraries use this a lot: you write a small class, and the parent class adds the machinery.

Decorators

```
@rx.event
def click_left(self):
    ...
```

A **decorator** is the `@something` line above a function. It is a short way to write:

```
def click_left(self):
    ...
click_left = rx.event(click_left)
```

The function is passed to `something`, and the name now points to whatever `something` returned. Libraries use decorators to *register* a function ("this function is an event handler", "this function answers `GET /api/notes`") or to change how it behaves.

Typed dictionaries and dataclasses

For data with a fixed shape, two tools describe the shape so that Pylance can check it:

```
from typing import TypedDict
from dataclasses import dataclass


class Part(TypedDict):          # a dict that must have these keys
    name: str
    exercises: int


@dataclass
class Note:                      # a small class that just holds data
    id: str
    content: str
    important: bool


p: Part = {"name": "State", "exercises": 14}
p["name"]                        # read like a dict
```

```
n = Note(id="1", content="HTML is easy", important=True)
n.content                        # read like an object
```

- A **TypedDict** is still an ordinary dict at runtime. It only tells Pylance which keys exist and what types their values have, so a typo like `p["nmae"]` is flagged.
- `@dataclass` writes `__init__` (and a readable `print` output, and == that compares the fields) for you, based on the annotated fields.

Exceptions and tracebacks

When something goes wrong while Python runs, it doesn't just stop: it **raises an exception**, an object that describes the problem. Its type names the kind of problem (`KeyError`, `TypeError`, `ZeroDivisionError`, ...) and it carries a message.

```
parts = {"name": "State"}
parts["exercises"]           # KeyError: 'exercises'
```

An exception travels *up* through the functions that called each other, until some code **catches** it. If nothing catches it, the program stops and prints a **traceback**: the chain of calls that led there, oldest at the top, and the exception at the very bottom.

```
Traceback (most recent call last):
  File "app.py", line 9, in <module>
    show(parts)
  File "app.py", line 5, in show
    print(parts["exercises"])
          ~~~~~^^^^^^^^^^^^^
KeyError: 'exercises'
```

Read a traceback **from the bottom up**: the last line says what happened, and the line above it says where, in which file and on which line. The lines higher up show how the program got there.

Catching an exception lets your code react instead of stopping:

```
try:
    count = parts["exercises"]
except KeyError:
    count = 0
```

The code in `try` runs normally. If an exception of the listed type is raised inside it (or inside any function it calls), Python jumps straight to the `except` block; the rest of the `try` block is skipped. Exceptions of other types keep travelling up.

Raising one yourself is how your own code reports a problem:

```
def average(numbers: list[float]) -> float:
    if len(numbers) == 0:
        raise ValueError("can't average an empty list")
    return sum(numbers) / len(numbers)
```

A `finally` block runs whether or not an exception happened, which makes it the place for cleanup that must always happen:

```
try:
    work()
finally:
    print("always printed, even if work() raised an exception")
```

Libraries use exceptions all the time to report failures: httpx raises one for a failed request (chapter 2), FastAPI turns one into an error response (chapter 3), and a test fails because an exception was raised (chapter 4).

with blocks

Some things must be undone after use: a file must be closed, a network connection released. Forgetting it, or skipping it because an exception jumped past the cleanup line, leaks resources. The `with` statement makes the cleanup automatic:

```
with open("notes.txt") as f:
    text = f.read()
# here the file is already closed, even if read() raised an exception
```

How it works: the object after `with` (here, the file returned by `open(...)`) is a **context manager**, an object with two special methods. Python calls the first one, `__enter__`, when the block starts; what it returns is bound to the name after `as`. When the block ends, for any reason, normally or through an exception, Python calls the second one, `__exit__`, which does the cleanup. It's a `try/finally` packaged inside the object. You'll meet `with` for network clients in chapter 2 and database sessions in chapter 3.

Generators and yield

A normal function runs from start to `return` in one go. A function that contains `yield` is different: it's a **generator function**, and it can pause in the middle.

```
def three_steps():
    print("step 1")
    yield "a"
    print("step 2")
    yield "b"
    print("step 3")


gen = three_steps()   # nothing is printed: the body hasn't started
next(gen)             # prints "step 1", then pauses at the first yield and returns "a"
next(gen)             # continues after that yield: prints "step 2", returns "b"
next(gen)             # prints "step 3", reaches the end: raises StopIteration
```

How it works:

- Calling a generator function doesn't run it. It returns a **generator object**, a paused copy of the function.
- Each `next(gen)` resumes the body until the next `yield`, hands out the value after `yield`, and freezes again, keeping all its local variables.
- When the body ends, the generator raises `StopIteration` to say "nothing more". A `for` loop calls `next` for you and stops at `StopIteration`, so `for x in three_steps():` gets `"a"`

and `"b"`.

Generator expressions, `(n * 10 for n in numbers)`, are a short way to write a generator. Chapter 3 uses a generator function in a clever way: code before `yield` sets something up, the yielded value is used by someone else, and code after `yield` cleans up when the generator is resumed.

Modules and imports

Every `.py` file is a **module**, and a folder of modules is a **package**. `import random` loads a module; `from random import randint` takes one name out of it. Python finds modules by searching a list of folders called `sys.path` (chapter 4a explains this in detail). Inside a package, a leading dot means "in this same package": `from .models import Note`.

Exercises 1.3–1.5

1.3: Course information, step 3

Move from separate variables to dicts. Each part becomes a dict with `name` and `exercises`, for example `part1 = {"name": "Fundamentals of Reflex", "exercises": 10}`. Pass the dicts to your components.

1.4: Course information, step 4

Put the parts in a list, `parts = [part1, part2, part3]`, and pass the whole list to `content` and `total`. In `content`, build the children with a list comprehension and pass them with `*`. In `total`, compute the sum from the list.

1.5: Course information, step 5

Make the course one dict, `{"name": ..., "parts": [...]}`, and pass it from `index`. Describe the shapes with `TypedDict` classes `Part` and `Course`, and use them in the type hints of your components.

c. Component state, event handlers

So far every page has been fixed. Interactive pages need data that changes while the app runs: a counter, a list of notes, the text in an input field. That data is the app's **state**.

A state class

```
import reflex as rx


class CounterState(rx.State):
    count: rx.Field[int] = rx.field(0)

    @rx.event
    def increase(self):
        self.count += 1
```

```
    @rx.event
    def reset(self):
        self.count = 0


def index() -> rx.Component:
    return rx.el.div(
        rx.el.div(CounterState.count),
        rx.el.button("plus", on_click=CounterState.increase),
        rx.el.button("zero", on_click=CounterState.reset),
    )


app = rx.App()
app.add_page(index)
```

- A state is a class that inherits from `rx.State`.
- `count: rx.Field[int] = rx.field(0)` declares a **state variable**, here an `int` that starts at 0. `rx.Field[int]` is the type hint Reflex reads to know what kind of value `count` holds; `rx.field(0)` gives the starting value.
- A method marked with `@rx.event` is an **event handler**: the only place where state may change. Inside it, `self.count` is a normal Python `int`.
- `on_click=CounterState.increase` connects the button's click event to the handler. Note that it has no parentheses: you pass the handler, you don't call it.

Each browser tab gets its own copy of the state. Open the page in two tabs and click in one: the other doesn't change. Reflex creates a separate state object per tab, keyed by a token the page sends when it connects.

What happens when you click

1. The browser sends a small message over the WebSocket: "event `increase` of `CounterState`, from tab X".
2. The Reflex backend finds that tab's state object and calls `increase(self)`. Your Python code runs on the server.
3. When the handler returns, Reflex checks which state variables changed and sends only those back to the browser, as a **delta**: `{"count": 1}`.
4. The React code in the browser updates its copy of `count`, and every part of the page that shows `count` redraws.

That is why the page updates without a reload, and also why every click is a small network round trip. Chapter 2b shows a case where this matters.

Vars: why CounterState.count is not a number

Inside a handler, `self.count` is an `int`. But in a component, `CounterState.count` is something else: a **Var**. Remember that component functions run *once*, at compile time, when there is no user

and no value yet. A Var is a placeholder that says "the value of `count`, whatever it is at the time". Reflex compiles it into a JavaScript expression that reads the current value in the browser.

You can see this in a Python shell with the app's state imported:

```
str(CounterState.count)        # something like 'reflex___..._counter_state.
    count_rx_state_'
str(CounterState.count + 1)    # '(reflex___...count_rx_state_ + 1)'
```

`CounterState.count + 1` doesn't add anything: it builds a new Var whose JavaScript adds 1 in the browser. This explains the rules for working with Vars in components:

- **Operators build Vars.** `+`, `-`, `*`, `/`, `==`, `>`, indexing `[i]` and attribute access work, and produce new Vars.
- **f-strings work.** `f"total is {CounterState.count}"` becomes a JavaScript template string. (Python formats the Var into a special marker that Reflex recognises when compiling.)
- **Python decisions don't work.** `if CounterState.count > 0:` would need the actual value at compile time, which doesn't exist. Reflex raises `VarTypeError: Cannot convert Var ... to bool for use with if, and, or, and not`. Use `rx.cond` instead (section 1d).
- **Built-in functions don't work.** `len(State.items)` raises `TypeError`, because Python's `len` needs a real list. Vars have their own methods instead: `State.items.length()`.

When an expression gets complicated, or needs Python functions like `max` or `list.index`, do it in Python with a computed var.

Computed vars

```
class ClickState(rx.State):
    left: rx.Field[int] = rx.field(0)
    right: rx.Field[int] = rx.field(0)

    @rx.var
    def total(self) -> int:
        return self.left + self.right
```

A method marked with `@rx.var` is a **computed var**: a value derived from other state variables. It runs on the server, as ordinary Python, so `if`, `len`, `max` and everything else are allowed. In a component you use it like any other Var, `ClickState.total`. Reflex recomputes it after every event and sends the new value to the browser when it changed.

The return type hint (`-> int`) is required: it is how Reflex knows the type of the Var it creates.

Use a computed var whenever a value can be *derived* from the state, instead of storing it separately. If you stored `total` as its own field, every handler would have to remember to update it, and sooner or later one would forget.

Passing handlers to child components

Handlers can be passed to your own components like any other value:

```
def button(text: str, on_click) -> rx.Component:
    return rx.el.button(text, on_click=on_click)
```

```
def index() -> rx.Component:
    return rx.el.div(
        ClickState.left,
        button("left", ClickState.click_left),
        button("right", ClickState.click_right),
        ClickState.right,
    )
```

`button` doesn't know anything about `ClickState`. It just attaches whatever handler it was given. Components that receive both data and handlers through parameters are reusable, which is the point of splitting a page into components.

d. A more complex state, debugging Reflex apps

Several pieces of state

A state class can hold as many variables as you need, of any JSON-compatible type: numbers, strings, booleans, lists, dicts, dataclasses.

```
class ClickState(rx.State):
    left: rx.Field[int] = rx.field(0)
    right: rx.Field[int] = rx.field(0)
    all_clicks: rx.Field[list[str]] = rx.field(default_factory=list)

    @rx.event
    def click_left(self):
        self.all_clicks.append("L")
        self.left += 1

    @rx.event
    def click_right(self):
        self.all_clicks.append("R")
        self.right += 1
```

Why default_factory. `rx.field(default_factory=list)` means "call `list()` to make a *new* empty list whenever a new state object is created". Writing `rx.field([])` would create one list when the class is defined; the rule borrowed from Python's dataclasses is that mutable defaults (lists, dicts) are made with a factory so that no two objects can end up sharing one. This book uses `default_factory` for every list and dict.

Changing a list in place. `self.all_clicks.append("L")` changes the list without assigning a new one. Reflex still notices: inside a handler, a state's lists and dicts are wrapped in objects that record every change. So both styles work:

```
self.all_clicks.append("L")                # change in place
self.all_clicks = self.all_clicks + ["L"]  # build a new list and assign it
```

Conditional rendering with rx.cond

Since `if` can't test a Var, Reflex has a component that decides in the browser:

```
from collections.abc import Sequence


def history(all_clicks: rx.vars.ArrayVar[Sequence[str]]) -> rx.Component:
    return rx.cond(
        all_clicks.length() == 0,
        rx.el.div("the app is used by pressing the buttons"),
        rx.el.div("button press history: ", all_clicks.join(" ")),
    )
```

`rx.cond(condition, a, b)` shows `a` when the condition is true and `b` otherwise. Both `a` and `b` are compiled into the page; the browser picks one each time the state changes. The third argument is optional: without it, nothing is shown when the condition is false.

- `all_clicks.length()` and `all_clicks.join(" ")` are Var methods, the browser-side versions of `len(...)` and `" ".join(...)`.
- A Var can be used directly as a condition: an empty string, `0`, an empty list and `None` count as false. `rx.cond(State.error_message, ...)` shows something only when there is a message.
- To combine conditions, use `&` (and), `|` (or) and `~` (not), with parentheses: `(items.length() >= 2) & (items.length() <= 10)`. The words `and`/`or`/`not` would need a Python `bool` and raise `VarTypeError`.

About the type hint. A list state variable becomes an `rx.vars.ArrayVar`. Reflex describes its contents with the wider type `Sequence` rather than `list`, so Pylance expects `ArrayVar[Sequence[str]]`. Hints on component parameters are only for Pylance; they don't change what the code does.

Debugging

Your code now runs in two places, and each has its own tools.

Handlers and computed vars run on the server. `print(...)` inside a handler prints in the terminal where `reflex run` runs, not in the browser. Errors in handlers show up there as a traceback: read it from the bottom, which names the exception, upwards to the line in your file.

Component functions run once, at compile time. A `print` in a component prints once, when the app compiles, and shows Vars as their JavaScript names, not as values. If a component raises an error, the app doesn't start at all; the traceback is again in the terminal.

The browser has developer tools. Press F12. The *Console* tab shows JavaScript errors, and the *Network* tab shows every request; the WebSocket connection is listed there, and its *Messages* view shows the events going up and the deltas coming down.

Keep the terminal and the browser console visible while you work, and add prints generously when something doesn't behave. The original course says it as a rule: *the first thing to do when the code doesn't work is to find out what the values actually are.*

Exercises 1.6–1.14

1.6: unicafe, step 1

Create `chapter1/unicafe`: a feedback app for a café. It has three buttons, `good`, `neutral` and `bad`, and shows how many times each was clicked. Keep the three counts in a state class, with one handler per button.

1.7: unicafe, step 2

Also show the total number of votes, the average score (good $= 1$, neutral $= 0$, bad $= -1$) and the percentage of positive votes. Use computed vars. Make sure nothing divides by zero before the first vote.

1.8: unicafe, step 3

Move the statistics into their own component, `statistics`.

1.9: unicafe, step 4

Show the statistics only after the first vote. Before that, show the text `no feedback given`. Use `rx.cond`.

1.10: unicafe, step 5

Extract two more components: `button`, which receives a text and a handler, and `statistic_line`, which shows one statistic (for example "good 6"). `statistics` uses `statistic_line` for each line.

1.11*: unicafe, step 6

Show the statistics in an HTML table: `rx.el.table`, `rx.el.tbody`, and one `rx.el.tr` row per statistic with two `rx.el.td` cells. Show the positive percentage with a `%` sign.

1.12*: anecdotes, step 1

Create `chapter1/anecdotes` with this list at the top of the file:

```
ANECDOTES = [
    "If it hurts, do it more often.",
    "Adding manpower to a late software project makes it later!",
    "Any fool can write code that a computer can understand. Good programmers write
    code that humans can understand.",
    "Premature optimization is the root of all evil.",
    "Debugging is twice as hard as writing the code in the first place. Therefore, if
     you write the code as cleverly as possible, you are, by definition, not smart
    enough to debug it.",
    "The only way to go fast, is to go well.",
]
```

Show one anecdote and a button `next anecdote` that picks a random one. `random.randint(a, b)` returns a random integer between `a` and `b`, both included. Keep the index of the selected anecdote in the state, and show the anecdote through a computed var.

1.13*: anecdotes, step 2

Add a `vote` button and show how many votes the shown anecdote has. Keep the votes in a list of numbers in the state, one per anecdote, starting at zero. (`[0 for _ in range(len(ANECDOTES))]` builds such a list; `_` is the conventional name for a loop variable you don't use.)

1.14*: anecdotes, step 3

Also show the anecdote with the most votes. If several share the top, showing any one of them is fine. `max(list)` returns the biggest item and `list.index(value)` returns the position of the first item equal to `value`; both need a computed var.

Chapter 2

Communicating with a server

Sections: a. Rendering a collection, modules · b. Forms · c. Getting data from a server · d. Altering data on the server · e. Adding styles to a Reflex app

a. Rendering a collection, modules

Two kinds of lists

A Reflex page can show a list in two different ways, and which one you need depends on *when* the list is known.

A fixed list, known when the app compiles. Use a normal list comprehension and pass the components with *:

```
def content(parts: list[Part]) -> rx.Component:
    return rx.fragment(
        *[part(p) for p in parts],
    )
```

This runs once, at compile time. Python builds three `part` components, and they are compiled into the page like any other component. That only works because `parts` is real Python data at that moment.

A list in the state, which changes while the app runs. A state list is a Var, and a comprehension can't loop over it: at compile time there are no items yet. Use `rx.foreach`:

```
def index() -> rx.Component:
    return rx.el.ul(
        rx.foreach(
            NoteState.notes,
            lambda n: rx.el.li(n.content, key=n.id),
        ),
    )
```

`rx.foreach(var, function)` calls your function *once*, at compile time, with a Var that stands for "one item of the list". The component it returns is a template. Reflex compiles it into a JavaScript loop that runs in the browser and repeats the template for every item the list holds at that moment. That's why `n.content` works: `n` is an object Var, and `.content` builds a Var for that field.

Keys. `key=n.id` gives each list element a stable identity. When the list changes, React compares the old and new elements by key to decide which DOM elements to keep, move or remove. Use

a value that is unique and stays the same for an item, such as its id. Using the item's position as the key breaks down when items are removed or reordered: a different item would get the old item's key.

reduce

`sum(...)` covers adding numbers. The general tool for "combine a list into one value" is `functools.reduce`:

```
from functools import reduce

total = reduce(lambda acc, p: acc + p["exercises"], parts, 0)
```

`reduce(function, items, start)` keeps a running value, the *accumulator*. It starts at `0`, then for each part calls `function(acc, part)` and keeps the result. For three parts with 10, 7 and 14 exercises, the accumulator goes 0, 10, 17, 31. In Python, `sum(p["exercises"] for p in parts)` is clearer for this case, but `reduce` is worth recognising.

Splitting an app into modules

As components multiply, one file gets long. Split it by job:

```
notes/
├── rxconfig.py
└── notes/
    ├── __init__.py
    ├── notes.py            ← the state and the page (index)
    ├── models.py           ← data shapes (dataclasses)
    └── components/
        ├── __init__.py
        └── note.py         ← one component per file
```

A file imports from another file in the same app with a **relative import**:

```
# in notes/notes.py
from .components.note import note
from .models import Note

# in notes/components/note.py
from ..models import Note
```

One dot means "the package this file is in", two dots "the package above it". The empty `__init__.py` files mark each folder as a package. Relative imports work because Reflex imports your app as a package (`notes.notes`); the same file run on its own with `python notes.py` would fail with "attempted relative import with no known parent package".

Exercises 2.1–2.5

2.1: Course information, step 6

Copy your course information app to `chapter2/courseinfo`. Give each part an `id`, and move the course display into one component, `course_view`, which shows the course name and its parts. `index` only calls `course_view(course)`. Add a fourth part and check that it appears.

2.2: Course information, step 7

Show the total number of exercises under the parts, in bold (`rx.el.strong`).

2.3*: Course information, step 8

Compute the total with `functools.reduce`.

2.4: Course information, step 9

Support any number of courses. `index` defines a list of courses, each with its own parts, and shows all of them under a main heading. Add a second course with two parts.

2.5: Separate module

Move `course_view` and the components it uses into `components/course.py`, together with the `Part` and `Course` types, and import them in the main file.

b. Forms

An input that follows the state

An input field has its own text, inside the browser. To use what the user types, you keep the text in the state and connect the two directions:

```
class NoteState(rx.State):
    notes: rx.Field[list[Note]] = rx.field(default_factory=list)
    new_note: rx.Field[str] = rx.field("a new note...")

    @rx.event
    def set_new_note(self, value: str):
        self.new_note = value


def index() -> rx.Component:
    return rx.el.input(value=NoteState.new_note, on_change=NoteState.set_new_note)
```

- `value=NoteState.new_note`: the input always shows what the state holds. The state is the single source of truth; this is called a **controlled** input.
- `on_change=NoteState.set_new_note`: every time the text changes, Reflex calls the handler with the new text. `on_change` passes one argument, the input's current value, so the handler takes `value: str`.

Without `on_change`, the input would show the state's value and refuse every keystroke, because each re-render would put the old value back.

Submitting a form

```
import typing


class NoteState(rx.State):
    ...

    @rx.event
    def add_note(self, form_data: dict[str, typing.Any]):
        self.notes.append(Note(id=str(len(self.notes) + 1), content=self.new_note,
    important=False))
        self.new_note = ""


def index() -> rx.Component:
    return rx.el.form(
        rx.el.input(value=NoteState.new_note, on_change=NoteState.set_new_note),
        rx.el.button("save", type="submit"),
        on_submit=NoteState.add_note,
    )
```

- A button with `type="submit"` inside a `form` submits it, and so does pressing Enter in one of its inputs.
- In plain HTML, submitting a form makes the browser send it to a server and *reload the page*. Reflex's form stops that default behaviour (it calls the browser's `preventDefault()` for you) and calls `on_submit` instead.
- `on_submit` passes one argument: a dict with the values of every input that has a `name` prop. Because our input is controlled, the text is already in `self.new_note`, so `form_data` goes unused. The parameter still has to be there, because Reflex passes it.
- `self.new_note = ""` empties the input, since the input shows whatever the state holds.

Filtering with a computed var

```
class NoteState(rx.State):
    ...
    show_all: rx.Field[bool] = rx.field(True)

    @rx.event
    def toggle_show_all(self):
        self.show_all = not self.show_all

    @rx.var
    def notes_to_show(self) -> list[Note]:
        if self.show_all:
            return self.notes
        return [n for n in self.notes if n.important]
```

The page loops over `NoteState.notes_to_show` instead of `NoteState.notes`. The full list stays untouched; the computed var derives the visible part from it after every event. The button's label changes with `rx.cond`:

```
rx.el.button(
    "show ",
    rx.cond(NoteState.show_all, "important", "all"),
    on_click=NoteState.toggle_show_all,
)
```

Typing feels slow: debouncing

Remember from 1c that every event is a round trip to the Reflex backend. A controlled input sends an event for *every keystroke*, and the input only shows the new character after the reply arrives. On a deployed app, with real network delay, fast typing lags behind and can even lose characters.

`rx.debounce_input` fixes this. It wraps an input, keeps the typing inside the browser, and sends `on_change` only after the user pauses:

```
rx.debounce_input(
    rx.el.input(value=NoteState.new_note, on_change=NoteState.set_new_note),
    force_notify_by_enter=True,
    force_notify_on_blur=True,
)
```

- By default it waits 300 ms after the last keystroke (`debounce_timeout=300`).
- `force_notify_by_enter=True` sends the value immediately when Enter is pressed. Without it, pressing Enter right after typing could submit the form before the last characters reached the state.
- `force_notify_on_blur=True` sends it immediately when the input loses focus, for example when the user clicks the save button.

Use it for every text input that is connected to the state.

Telling the user something: rx.window_alert

A handler can return an *event* for the browser to run. `rx.window_alert(message)` is one: it opens the browser's alert box.

```
@rx.event
def add_note(self, form_data: dict[str, typing.Any]):
    if self.new_note == "":
        return rx.window_alert("a note can't be empty")
    ...
```

`return` ends the handler there, so nothing is added. Section 2d and 2e show more events a handler can return.

Exercises 2.6–2.10

Create `chapter2/phonebook`. Start from this page:

```
class PhonebookState(rx.State):
    persons: rx.Field[list[Person]] = rx.field(default_factory=list)
    new_name: rx.Field[str] = rx.field("")
```

with a `Person` dataclass in `models.py` (start with only `name: str`), a heading "Phonebook", a form with a name input and an `add` button, and a list of names under the heading "Numbers".

2.6: The Phonebook, step 1

Adding a name with the form adds it to the list on the page.

2.7: The Phonebook, step 2

Prevent duplicates. If the name already exists, show an alert (`rx.window_alert`) such as "Maren Holt is already added to phonebook" and don't add it. `any(...)` with a generator expression checks whether at least one item matches: `any(p.name == self.new_name for p in self.persons)`.

2.8: The Phonebook, step 3

Add a phone number: a `number` field in `Person`, a second input, and the number shown next to each name.

2.9*: The Phonebook, step 4

Add a filter input. Only persons whose name contains the filter text are shown, ignoring upper and lower case (`str.lower()`). Use a computed var. Start the state with a few persons so there is something to filter:

```
Person(name="Maren Holt", number="040-123456"),
Person(name="Idris Okafor", number="39-44-5323523"),
Person(name="Lena Vasquez", number="12-43-234345"),
Person(name="Tobias Rhee", number="39-23-6423122"),
```

2.10: The Phonebook, step 5

Split the page into components: the filter, the form for adding a person, and the list of persons (with a component for one person). Keep the state in its own file, `state.py`. Pass Vars and handlers to the components through parameters. Use `rx.debounce_input` for every input.

c. Getting data from a server

So far all data has lived in the state, which disappears when the Reflex server restarts, and which each browser tab has its own copy of. Real apps keep their data on a **server** that every user talks to.

HTTP in short

Browsers and servers talk with **HTTP**. Every exchange is a **request** followed by a **response**.

A request has:

- a **method**, which says what to do: `GET` (read), `POST` (create), `PUT/PATCH` (change), `DELETE` (remove);
- a **URL**, which says what it's about: `http://localhost:3001/notes/2` is the scheme (`http`), the host (`localhost`, which means "this computer"), the **port** (`3001`, which program on that computer) and the path (`/notes/2`);
- **headers**, extra information as name–value pairs, such as `Content-Type: application/json`;
- an optional **body**, the data sent along, for example the new note.

A response has a **status code**, headers, and an optional body. The status code's first digit gives the category:

Code	Meaning
`200 OK`	success, here is the result
`201 Created`	success, a new resource was created
`204 No Content`	success, nothing to send back
`400 Bad Request`	the request itself is wrong
`404 Not Found`	no such resource
`500 Internal Server Error`	the server crashed while handling it

2xx means success, 4xx means the client made a mistake, and 5xx means the server failed.

JSON

Data travels as text. The common format is **JSON** (JavaScript Object Notation), which looks almost like Python's literals:

```
[
  {"id": "1", "content": "HTML is easy", "important": true},
  {"id": "2", "content": "Browser can execute only JavaScript", "important": false}
]
```

JSON has objects (Python dicts), arrays (lists), strings, numbers, `true/false` (`True/False`) and `null` (`None`). Turning Python data into JSON text is *serialising*; reading it back is *parsing*.

REST

Most web APIs follow a convention called **REST**. Data is organised into **resources**, each with its own URL, and the HTTP method says what to do with it:

URL	Method	What it does
`/notes`	`GET`	return all notes
`/notes/10`	`GET`	return the note with id 10
`/notes`	`POST`	create a new note from the request body
`/notes/10`	`PUT`	replace note 10 with the request body
`/notes/10`	`PATCH`	change only the fields in the request body
`/notes/10`	`DELETE`	remove note 10

`/notes` is a *collection*; `/notes/10` is one item in it. The server, not the client, chooses the id of a new item.

A server to practise with

The original course uses a tool called *json-server*. This book uses a small Python replacement, `tools/mock_server.py`: it reads a JSON file and serves every top-level list in it as a REST collection, writing every change back to the file. Get the file from the course repository and put it in `tools/` at the root of yours. It uses only Python's standard library.

In `chapter2/notes`, create `db.json`:

```
{
  "notes": [
    {"id": "1", "content": "HTML is easy", "important": true},
    {"id": "2", "content": "Browser can execute only JavaScript", "important": false},
    {"id": "3", "content": "GET and POST are the most important methods of HTTP
    ↪  protocol", "important": true}
  ]
}
```

and start the server from that folder, in a second terminal:

```
python ../../tools/mock_server.py db.json --port 3001
```

Open `http://localhost:3001/notes` in the browser. The browser sends a `GET` request, and the JSON appears. The server prints every request it receives, which is useful when your app doesn't behave as expected. It picks new ids itself, as random strings like `"a3f9"`.

Synchronous and asynchronous code

A request to a server takes time: milliseconds locally, much longer over the internet. What should the program do while it waits?

Synchronous code simply waits. The line that sends the request doesn't return until the response is there, and nothing else happens in the meantime. For a server that serves many users, that is wasteful: while one handler waits for the network, others could run.

Asynchronous code lets the program do other work while waiting. In Python it is written with `async` and `await`:

```
import asyncio


async def fetch_notes():
    print("asking")
```

```
    await asyncio.sleep(1)      # stands for waiting on the network
    print("got them")
    return ["HTML is easy"]
```

How it works:

- `async def` defines a **coroutine function**. Calling it does *not* run the body; it returns a **coroutine**, a paused piece of work.
- An **event loop** runs coroutines. It runs one until it reaches an `await` on something that isn't ready yet. Then that coroutine is put aside, and the loop runs another one. When the awaited thing is ready (the response arrived, the second passed), the loop continues the first coroutine from exactly where it stopped.
- `await` can only be used inside an `async def`. Writing it is how a coroutine says "I'm waiting here; do something else meanwhile".

So there is still only one thread doing work at any moment; the loop just switches between tasks at every `await`. Reflex runs an event loop for you, and an event handler may be an `async def`: while it awaits a server's reply, the Reflex backend keeps handling other users' events.

httpx

Why a library is needed. An HTTP request is, underneath, text sent over a network connection. To send one by hand, a program has to open a connection to the right host and port, set up encryption for `https`, write the request line, the headers and the body in the exact format HTTP prescribes, read the reply as raw bytes, split it into status, headers and body, and handle redirects, timeouts and broken connections. Python's standard library has low-level tools for this (`urllib`, `http.client`), but using them takes many lines for every request.

httpx is a library that does all of that behind one function call, in the role axios plays in the original course. You say "GET this URL" and get back a response object with the status, the headers and the body, already decoded. It also supports `async`, which matters inside Reflex handlers, as explained above. Install it (`pip install httpx`) and add it to the app's `requirements.txt`.

```
import httpx


async def get_all() -> list[dict]:
    async with httpx.AsyncClient() as client:
        response = await client.get("http://localhost:3001/notes")
    response.raise_for_status()
    return response.json()
```

- `httpx.AsyncClient()` is the asynchronous client, which you `await`. httpx also has a synchronous one (`httpx.get(...)`) that would block the Reflex backend while waiting.
- `async with ... as client:` opens the client and guarantees it is closed when the block ends, even if an error happens. This is the `with` block from 1b: the client is a context manager whose cleanup closes its network connections. `async with` is the version whose setup and cleanup are coroutines, so Python awaits them.
- The `response = ...` line is inside the `with` block because the request needs the open client. The lines after it are outside: by then the whole response has been read into memory, so it can be used after the client is closed.

- `client.get(url)` sends a `GET`. There are matching `.post`, `.put`, `.patch` and `.delete`. To send a JSON body, pass `json={...}`: httpx serialises it and sets the `Content-Type` header.
- `response.raise_for_status()` raises `httpx.HTTPStatusError` if the status is 4xx or 5xx. Without it, a `404` would come back like any other response, and your code would carry on with an error body as if it were data.
- `response.json()` parses the body into Python lists and dicts.

Turning JSON into objects

The notes come back as dicts. The app works with `Note` objects, so convert them with `**`:

```
# notes/models.py
from dataclasses import dataclass


@dataclass
class Note:
    id: str
    content: str
    important: bool
```

```
[Note(**n) for n in response.json()]
```

`Note(**{"id": "1", "content": "HTML is easy", "important": True})` is `Note(id="1", content="HTML is easy", important=True)`. If the JSON has a key the dataclass doesn't, or lacks one it needs, this raises `TypeError`, which tells you right away that the server's data doesn't have the shape you expected.

Loading data when the page opens

```
class NoteState(rx.State):
    notes: rx.Field[list[Note]] = rx.field(default_factory=list)

    @rx.event
    async def load_notes(self):
        self.notes = await note_service.get_all()


app = rx.App()
app.add_page(index, on_load=NoteState.load_notes)
```

`on_load` is an event that runs every time the page is opened. The page first shows an empty list, then the notes appear when the handler finishes and the delta arrives. This is the Reflex version of React's `useEffect` in the original course.

Services: one module that talks to the server

Put every request in one module, so the state never deals with URLs or HTTP details:

```
# notes/services/note_service.py
import httpx
```

```
from ..models import Note

BASE_URL = "http://localhost:3001/notes"


async def get_all() -> list[Note]:
    async with httpx.AsyncClient() as client:
        response = await client.get(BASE_URL)
    response.raise_for_status()
    return [Note(**n) for n in response.json()]


async def create(content: str, important: bool) -> Note:
    async with httpx.AsyncClient() as client:
        response = await client.post(BASE_URL, json={"content": content, "important":
    important})
    response.raise_for_status()
    return Note(**response.json())
```

`services/` needs an empty `__init__.py` too. In the state, `from .services import note_service` and then `await note_service.get_all()`. When the server's address changes (and it will, in chapter 3), only `BASE_URL` changes.

Exercise 2.11

2.11: The Phonebook, step 6

Put the persons in `chapter2/phonebook/db.json` under the key `"persons"`, each with an `"id"`, and run the mock server on port 3001. Load the persons from the server when the page opens, using `httpx` and `on_load`. Add `id: str` to `Person`.

d. Altering data on the server

Creating

Saving a new note becomes a `POST`. The server answers with the note it saved, including the id it chose:

```
@rx.event
async def add_note(self, form_data: dict[str, typing.Any]):
    created = await note_service.create(content=self.new_note, important=random() < 0.5)
    self.notes.append(created)
    self.new_note = ""
```

Add the object the *server* returned, not one built locally: only the server's copy has the real id. (`random() < 0.5` makes about half of the new notes important, so there's something to filter.)

Changing

Each note gets a button that toggles its importance. A button inside `rx.foreach` needs to tell the handler *which* note was clicked:

```
# notes/components/note.py
def note(data: rx.vars.ObjectVar[Note], toggle_importance) -> rx.Component:
    return rx.el.li(
        data.content,
        " ",
        rx.el.button(
            rx.cond(data.important, "make not important", "make important"),
            on_click=lambda: toggle_importance(data.id),
        ),
        key=data.id,
    )
```

```
# notes/notes.py, in index()
rx.foreach(NoteState.notes_to_show, lambda n: note(n, NoteState.toggle_importance))
```

How the id reaches the handler. `on_click=NoteState.toggle_importance` alone would call the handler with no id. `lambda: toggle_importance(data.id)` is a function Reflex calls once, at compile time. Calling an event handler with arguments doesn't run it; it builds an *event spec*: "call `toggle_importance` with this Var as argument". `data.id` is the Var for the current item's id, so in the browser each button sends its own note's id along with the event.

The handler and the service:

```
@rx.event
async def toggle_importance(self, note_id: str):
    note = next(n for n in self.notes if n.id == note_id)
    updated = await note_service.update_important(note_id, not note.important)
    self.notes = [updated if n.id == note_id else n for n in self.notes]
```

```
async def update_important(note_id: str, important: bool) -> Note:
    async with httpx.AsyncClient() as client:
        response = await client.patch(f"{BASE_URL}/{note_id}", json={"important":
    important})
    response.raise_for_status()
    return Note(**response.json())
```

- `next(generator)` returns the first item the generator produces: here, the first note with that id. With no match it raises `StopIteration`; `next(generator, None)` returns `None` instead.
- `PATCH` sends only the field that changes. `PUT` would replace the whole note, so you'd have to send every field.
- The last line builds a new list in which the changed note is replaced by the server's version and every other note is kept.

Handling errors

Requests fail: the server may be down, or the note may have been deleted by someone else in the meantime. Then `raise_for_status()` raises `httpx.HTTPStatusError`, and if nobody catches it,

the handler stops with a traceback in the terminal and the user sees nothing.

```
@rx.event
async def toggle_importance(self, note_id: str):
    note = next(n for n in self.notes if n.id == note_id)
    try:
        updated = await note_service.update_important(note_id, not note.important)
    except httpx.HTTPStatusError as error:
        if error.response.status_code == 404:
            self.notes = [n for n in self.notes if n.id != note_id]
            return rx.window_alert(f"the note '{note.content}' was already deleted from
    the server")
        raise
    self.notes = [updated if n.id == note_id else n for n in self.notes]
```

- `try` runs the code; if an exception of the listed type happens inside it, Python jumps to `except`. `as error` gives you the exception object, and `error.response` is the response that caused it.
- For a `404`, the note is gone on the server, so it is removed locally too, and the user is told why.
- `raise` with nothing after it re-raises the same exception. Every status you don't handle on purpose stays visible in the terminal instead of being silently ignored. Catching an error and doing nothing is one of the worst bugs to find later.

Deleting, and asking first

Deleting is a `DELETE` request; the mock server answers with the removed item. Exercise 2.14 asks to confirm before deleting. The browser's confirm box is a JavaScript function, `window.confirm(message)`, which returns `true` or `false`. Reflex can run JavaScript and send the result to a handler:

```
import json


@rx.event
def ask_delete(self, person_id: str):
    person = next(p for p in self.persons if p.id == person_id)
    message = f"Delete {person.name}?"
    return rx.call_script(
        f"window.confirm({json.dumps(message)})",
        callback=lambda result: PhonebookState.delete_person(person_id, result),
    )


@rx.event
async def delete_person(self, person_id: str, confirmed: bool):
    if not confirmed:
        return
    ...
```

- `rx.call_script(code, callback=...)` is an event that runs `code` in the browser. When it finishes, its result is sent to the callback, here as `result`.

- `json.dumps(message)` turns the Python string into a JavaScript string literal with quotes and escaping, so a name containing a quote can't break the script.
- The callback is a lambda that builds the next event, passing along both the id (a normal Python value, captured when `ask_delete` runs) and the user's answer.

Exercises 2.12–2.15

2.12: The Phonebook, step 7

New persons are saved on the server with `POST`.

2.13: The Phonebook, step 8

Move every request into `services/person_service.py`.

2.14: The Phonebook, step 9

Add a delete button to each person. Ask for confirmation with `window.confirm` before deleting, and remove the person on the server and on the page.

2.15*: The Phonebook, step 10

If a name that already exists is added, ask "Maren Holt is already added to phonebook, replace the old number with a new one?". If confirmed, change the number on the server with `PATCH` and replace the person on the page with the server's version.

e. Adding styles to a Reflex app

CSS

CSS rules choose elements with a **selector** and set properties on them:

```
h1 {
    color: green;
    font-style: italic;
}

.note {
    color: grey;
    padding-top: 3px;
    font-size: 15px;
}
```

- `h1 { ... }` applies to every `<h1>` element.
- `.note { ... }` applies to every element with the **class** `note`. Classes let you style chosen elements instead of all elements of a type.

When several rules set the same property on one element, the more specific selector wins (a class beats a tag name), and between equally specific rules, the later one wins. That's the "cascade" in Cascading Style Sheets.

Stylesheets in Reflex

Put the rules in `assets/styles.css`. Everything in `assets/` is served as-is at the root of the site, so the file is available at `/styles.css`. Load it in the app:

```
app = rx.App(stylesheets=["/styles.css"])
```

Give an element a class with the `class_name` prop. It is called `class_name` because `class` is a reserved word in Python:

```
rx.el.li(data.content, class_name="note")
```

Inline styles set CSS properties directly on one component, as keyword props. CSS names with dashes are written with underscores:

```
def footer() -> rx.Component:
    return rx.el.div(
        rx.el.em("Note app, Department of Computer Science"),
        color="green",
        font_style="italic",
        font_size="16px",
    )
```

Inline styles are convenient for one-off cases. For anything reused, a class in the stylesheet keeps the look in one place.

An error message component

Replace the alert from 2d with a message on the page:

```
# notes/components/notification.py
def notification(message: rx.Var[str]) -> rx.Component:
    return rx.cond(
        message,
        rx.el.div(message, class_name="error"),
    )
```

```
.error {
    color: red;
    background: lightgrey;
    font-size: 20px;
    border-style: solid;
    border-radius: 5px;
    padding: 10px;
    margin-bottom: 10px;
}
```

An empty string counts as false, so the component shows nothing while `error_message` is `""`. The state gets `error_message: rx.Field[str] = rx.field("")`, and the 404 branch sets it instead of returning an alert.

Removing the message after five seconds: background events

The message should disappear by itself. The obvious version doesn't work well:

```
@rx.event
async def toggle_importance(self, note_id: str):
    ...
    self.error_message = "..."
    await asyncio.sleep(5)          # don't do this
    self.error_message = ""
```

Why not. Reflex processes the events of one browser tab *one at a time*: while a handler runs, it holds that tab's state, and the next event waits in a queue. A handler that sleeps for five seconds freezes the tab for five seconds; every click in the meantime waits. Also, the delta is only sent when the handler ends, so the message would never even appear.

A **background event** runs without holding the state:

```
import asyncio


class NoteState(rx.State):
    ...

    @rx.event(background=True)
    async def clear_error_later(self):
        await asyncio.sleep(5)
        async with self:
            self.error_message = ""
```

- `@rx.event(background=True)` marks it as a background event. It must be `async`.
- While it sleeps, other events of the same tab run normally.
- To change the state, it must take the state for a moment with `async with self:`. Inside the block it has the latest state and may change it; when the block ends, the changes are sent to the browser, and other events can run again. Changing `self` outside such a block raises an error.

Start it by *returning* it from the handler that sets the message:

```
if error.response.status_code == 404:
    self.notes = [n for n in self.notes if n.id != note_id]
    self.error_message = f"Note '{note.content}' was already removed from the server"
    return NoteState.clear_error_later
```

A handler can return another event (or a list of events), and Reflex runs it after the handler finishes. Here, the message is sent to the browser first, and five seconds later the background event clears it.

Data from an external API, and secrets

Exercises 2.18–2.20 use two public APIs. The second needs an **API key**, a secret string that identifies you to the service. A key must never be written in the code: anything in a Git repository

can end up public. Put it in an **environment variable** instead. Environment variables are name–value pairs that every program receives from the terminal that started it.

```
export WEATHER_API_KEY=your-key-here
reflex run
```

`export` sets the variable in the current terminal, for every program started from it. In Python, `os.environ` is a dict of all of them:

```
import os

API_KEY = os.environ["WEATHER_API_KEY"]                          # KeyError if it's missing
BACKEND = os.environ.get("BACKEND_URL", "http://localhost:3001")   # default if it's
    missing
```

With `[...]`, a missing key stops the app at once with a clear error, which is right for something the app can't work without. `export` lasts only until the terminal closes; chapter 3c shows how to keep such values in a file.

Backend-only state variables. The countries app loads all 250 countries but shows only a few. A state variable whose name starts with an underscore, such as `_all_countries`, is *backend-only*: it stays on the Reflex server and is never sent to the browser. Use one for data the page never shows directly; sending large data to the browser on every change would be slow for nothing.

Query parameters. Many APIs take options in the URL after a `?`, as `name=value` pairs joined with `&`: `...weather?q=Helsinki&units=metric`. httpx builds that part for you and escapes special characters:

```
await client.get(url, params={"q": city, "units": "metric"})
```

Exercises 2.16–2.20

2.16: Phonebook, step 11

After a person is added or a number changed, show a green success message for five seconds, such as "Added Maren Holt". Put the styles in `assets/styles.css`.

2.17*: Phonebook, step 12

Open the app in two browsers. Delete a person in one, then try to change the same person's number in the other. Show a red error message such as "Information of Maren Holt has already been removed from server" and remove the person from the page. Use one notification component for both kinds of message, with a class (`"success"` or `"failure"`) kept in the state.

2.18*: Data for countries, step 1

Create `chapter2/countries`. The API at `https://studies.cs.helsinki.fi/restcountries/api/all` returns all countries. Load them when the page opens, and show an input for searching by name:

- more than 10 matches: show "Too many matches, specify another filter";
- 2 to 10 matches: list their names;

- exactly one match: show its name, capital, area, languages and flag.

Not every country has a capital or languages; handle both. Keep the full list in a backend-only state variable.

2.19*: Data for countries, step 2

Add a `show` button next to each listed name that shows that country's details.

2.20*: Data for countries, step 3

Show the current weather in the capital: temperature, wind and the weather icon. Use OpenWeatherMap (`https://api.openweathermap.org/data/2.5/weather`, with the query parameters `q`, `appid` and `units=metric`). Create a free account for an API key and read it from the environment variable `WEATHER_API_KEY`. A new key can take a while to become active. Icons are at `https://openweathermap.org/img/wn/<icon>@2x.png`.

Chapter 3

Programming a server with FastAPI

Sections: a. FastAPI basics · b. Deploying the app to the internet · c. Saving data to PostgreSQL · d. Validation and Ruff

a. FastAPI basics

In chapter 2 the data lived on a fake server. Now you write the real one: a **backend** that stores the data and answers HTTP requests from any client. The original course uses Node.js with Express; this book uses **FastAPI**.

Why a separate backend

The Reflex app already has a server, so why another one?

- The Reflex server holds *per-tab* state. Data every user shares (all the notes) needs one central place.
- A backend with a REST API can be used by any client: your Reflex app, a phone app, another website, or a test (chapter 4).
- Each part can change on its own. The frontend only knows the API's URLs, not how the data is stored.

What a server program has to do

A backend is a program that waits for HTTP requests and answers them. Look at everything it has to do for *every* request:

1. listen on a network port and accept the incoming connection;
2. read the raw bytes and split them into method, path, headers and body;
3. decide which piece of code should answer this method and path (**routing**);
4. pull values out of the path and the URL, and parse the JSON body into Python data;
5. check that the data is complete and has the right types, and answer with an error if not;
6. run your actual logic;
7. turn the result back into JSON, pick the status code, write the headers, and send the bytes back.

Only step 6 is specific to your app. The mock server from chapter 2 (`tools/mock_server.py`) does steps 2–7 by hand with Python's standard library: it splits the path on `/` itself, reads the body length from a header, calls `json.loads`, and writes every header line with `send_header`. It's about 150 lines, and it doesn't validate anything.

FastAPI is a **web framework**: it does steps 2–5 and 7 for you. You write one Python function per URL and method, and describe with type hints what data each function expects and returns. FastAPI reads those hints and does the parsing, the checking, the error responses and the JSON conversion. It is built from two other libraries:

- **Starlette** handles the web part: requests, responses, routing and middleware.
- **Pydantic** handles the data part: checking and converting data according to type hints. You'll use it directly in this chapter.

Step 1, listening on the network, is done by a separate program, **uvicorn**. The next sections show how the two connect.

Installing and running

```
pip install fastapi uvicorn
```

Create `chapter3/notes_backend/main.py`:

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def root() -> dict:
    return {"message": "Hello World!"}
```

and run it from that folder:

```
uvicorn main:app --reload --port 3001
```

- `main:app` means "import the module `main` (the file `main.py`) and use the variable `app` in it".
- `--reload` restarts the server whenever a `.py` file changes. Use it only while developing.
- `--port 3001` chooses the port. Port 3000 is Reflex's.

Open `http://localhost:3001/`: the browser shows `{"message": "Hello World!"}`. FastAPI turned the returned dict into JSON.

How uvicorn and FastAPI fit together: ASGI

Two programs are involved, each with one job:

- **uvicorn** handles the network: it opens the port, reads the raw bytes of each request and writes the response bytes back.
- **Your FastAPI app** handles the logic: which function answers which URL, validation, your data.

ASGI (Asynchronous Server Gateway Interface) is the agreement between them. When a request arrives, uvicorn turns the bytes into a plain Python dict, roughly

```
{"type": "http", "method": "GET", "path": "/api/notes", "headers": [...]}
```

and calls your app with it. The app answers by sending back Python data too: the status, the headers, the body. uvicorn turns that into bytes and sends them to the client.

It works like a plug and a socket: any ASGI server can run any ASGI app. And because the app never touches the network itself, other code can call it directly with such a dict, which is exactly what tests do in chapter 4b.

Routes

```
@app.get("/api/notes")
def get_notes() -> list[Note]:
    return notes
```

`@app.get("/api/notes")` is a decorator (1b) that *registers* the function: "when a `GET` request for `/api/notes` arrives, call this function". Such a function is a **route** (also called a path operation). There is one decorator per HTTP method: `@app.post`, `@app.put`, `@app.patch`, `@app.delete`. Whatever the function returns is sent as JSON with status `200`.

The `/api` prefix is a convention that separates the data API from any pages the server might also serve.

Data with Pydantic

For now the notes live in a Python list in memory. Each note is a **Pydantic model**:

```
from pydantic import BaseModel, Field


class Note(BaseModel):
    id: str
    content: str
    important: bool


notes: list[Note] = [
    Note(id="1", content="HTML is easy", important=True),
    Note(id="2", content="Browser can execute only JavaScript", important=False),
    Note(id="3", content="GET and POST are the most important methods of HTTP protocol",
     important=True),
]
```

What Pydantic is for. Data that comes from outside your program can't be trusted: a JSON body may lack a field, have a number where text should be, or contain `null`. Type hints alone don't help, because Python ignores them at runtime (1b). Checking every field by hand ("is it there? is it a string? is it long enough?") would be long and repetitive. **Pydantic** turns type hints into real checks: you describe the shape of the data once, as a class, and Pydantic checks every object against it.

How a BaseModel works. A class that inherits from `BaseModel` looks like a dataclass. The difference happens when you create an object: for every field, Pydantic takes the value you passed and checks it against the field's type and rules. Three outcomes are possible:

- the value already has the right type: it is kept;
- it has a different type that can be converted *without losing information*: it is converted;
- anything else: the field fails.

These are real results for a model with `n: int`, `ok: bool` and `s: str`:

Field	Value passed	Result
`n: int`	`"5"`	`5`: a string of digits is converted
`n: int`	`5.0`	`5`: a float with nothing after the point is converted
`n: int`	`5.5`	error `int_from_float`: converting would lose the `.5`
`n: int`	`"5.5"`	error `int_parsing`
`ok: bool`	`"yes"`	`True`: common words like yes/no, true/false, 1/0 are converted
`ok: bool`	`"maybe"`	error `bool_parsing`
`s: str`	`5`	error `string_type`: numbers are *not* turned into text

All errors at once. Pydantic doesn't stop at the first problem. It checks every field and, if any failed, raises one `ValidationError` that lists all of them, each with the field's name (`loc`), an error type and a message. That's why FastAPI's error responses (chapter 3d) contain a list.

```
from pydantic import ValidationError

try:
    Note(id="1", content=None, important="maybe")
except ValidationError as error:
    print(error.error_count())    # 2: content and important both failed
```

The other direction. A model object turns back into plain data with `.model_dump()` (a dict) or `.model_dump_json()` (a JSON string). FastAPI uses this when it sends a model in a response.

How FastAPI uses Pydantic. Incoming: a route argument typed with a model is built from the request's JSON, and a `ValidationError` becomes a `422` response. Outgoing: the route's return type is used to check and convert what the route returned, which is then dumped to JSON.

Return type hints shape the response. `-> list[Note]` is not only for Pylance: FastAPI reads it, checks what the function returned against it, keeps only the fields the model declares, and turns the result into JSON. This becomes important in chapter 4, where a user has a password hash that must never be sent.

One item: path parameters and 404

```
from fastapi import HTTPException


@app.get("/api/notes/{note_id}")
def get_note(note_id: str) -> Note:
    note = next((n for n in notes if n.id == note_id), None)
```

```
    if note is None:
        raise HTTPException(status_code=404, detail="note not found")
    return note
```

- `{note_id}` in the path is a **path parameter**. FastAPI takes that part of the URL and passes it to the function argument with the same name. For `/api/notes/2`, `note_id` is `"2"`.
- The argument's type hint is used for conversion and checking. With `note_id: int`, a request for `/api/notes/abc` would be rejected before your function runs (chapter 3c does this).
- `raise HTTPException(status_code=404, detail="...")` stops the route and sends that status with the body `{"detail": "note not found"}`. It is an exception, so it works from any depth of function calls.

Deleting

```
@app.delete("/api/notes/{note_id}", status_code=204)
def delete_note(note_id: str) -> None:
    global notes
    notes = [n for n in notes if n.id != note_id]
```

- `status_code=204` sets the status for a successful response: "No Content". The response has no body.
- The comprehension builds a new list without the deleted note. Assigning to `notes` inside a function would normally create a new *local* variable; `global notes` says "I mean the module-level `notes`". (Changing the list in place, like `notes.append(...)`, doesn't need `global`, because it doesn't assign the name.)
- Deleting a note that doesn't exist also answers `204`. The goal, "this note doesn't exist", is reached either way. Answering `404` would also be defensible; what matters is being consistent.

Creating: request bodies

```
class NoteCreate(BaseModel):
    content: str = Field(min_length=1)
    important: bool = False


def generate_id() -> str:
    max_id = max((int(n.id) for n in notes), default=0)
    return str(max_id + 1)


@app.post("/api/notes", status_code=201)
def create_note(data: NoteCreate) -> Note:
    new_note = Note(id=generate_id(), content=data.content, important=data.important)
    notes.append(new_note)
    return new_note
```

- A route argument typed with a Pydantic model is read from the **request body**. FastAPI parses the JSON, builds a `NoteCreate` from it, and only then calls your function.
- If the body breaks a rule (`content` missing, empty, or not a string), your function never runs. FastAPI answers `422 Unprocessable Content` with a list of what was wrong (chapter 3d looks at this body in detail).
- `Field(min_length=1)` attaches a rule to a field, on top of its type. `important: bool = False` gives a default, which makes the field optional.
- `NoteCreate` has no `id`: the client must not choose it. Having separate models for what comes *in* and what goes *out* is a pattern you will use for every resource.
- `max(..., default=0)` returns 0 for an empty list instead of raising `ValueError`.

Where FastAPI gets each argument from

A request carries data in three places: the **path** (`/api/notes/5`), the **query string** after a `?` (`?q=hi&limit=3`, pairs of `name=value` joined with `&`), and the **body**. You never tell FastAPI where to look. It decides for each argument of your function by these rules, in this order:

1. The argument's name appears in the path as `{name}`: it's a **path parameter**.
2. Its type is a Pydantic model: it's read from the **body**.
3. Its default is `Depends(...)`: FastAPI calls that function and passes the result (chapter 3c).
4. Its type is `Request`: FastAPI passes the raw request object (see Middleware below).
5. Anything else with a simple type (`int`, `str`, `bool`, ...): it's a **query parameter**. With a default value it's optional; without one it's required.

```
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None, limit: int = 10):
    return {"item_id": item_id, "q": q, "limit": limit}
```

Request	Result
`GET /items/5?q=hi&limit=3`	`{"item_id": 5, "q": "hi", "limit": 3}`
`GET /items/5`	`{"item_id": 5, "q": null, "limit": 10}`: defaults used
`GET /items/5?limit=abc`	`422`, with `"loc": ["query", "limit"]`

Every value in a URL is text. FastAPI converts it to the declared type with Pydantic's rules, which is why `"5"` becomes the integer `5`, and why a value that can't be converted gives a `422`. The `loc` in the error says where the bad value was: `"path"`, `"query"` or `"body"`.

The interactive docs

Open `http://localhost:3001/docs`. FastAPI builds this page from your routes and models: every route, its parameters, the body it expects and the responses. How: FastAPI collects everything it knows about your routes into one JSON document in a standard format called **OpenAPI**, served at `/openapi.json`. The `/docs` page is a ready-made web app (Swagger UI) that reads that document and draws the forms. **Try it out** sends real requests. This replaces the Postman or REST client used in the original course, and it's always up to date because it is generated from the code.

Returning HTML

A route can also return a page instead of JSON:

```
from fastapi.responses import HTMLResponse


@app.get("/", response_class=HTMLResponse)
def root() -> str:
    return "<h1>Hello World!</h1>"
```

`response_class=HTMLResponse` sends the string as-is with `Content-Type: text/html`, so the browser shows it as a page.

Middleware

A **middleware** is a function that runs for *every* request, around whichever route handles it. It is the place for things that apply to everything, such as logging:

```
import time

from fastapi import Request


@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    print(request.method, request.url.path, response.status_code, f"{ms:.1f} ms")
    return response
```

- Code before `await call_next(request)` runs before the route; `call_next` runs the matching route and returns its response; code after it runs after the route.
- `time.perf_counter()` is a clock meant for measuring durations.
- The middleware must be `async def`, because `call_next` is awaited.

The Request object. `request` is an object Starlette builds from the incoming request, before any route runs. It gives you everything the client sent:

Attribute	For POST /x?a=1 with a JSON body
`request.method`	`"POST"`
`request.url.path`	`"/x"`
`request.url.query`	`"a=1"`
`request.query_params`	a dict-like object, `{"a": "1"}`
`request.headers`	a dict-like object, `request.headers["content-type"]` is `"application/json"`
`request.client.host`	the client's address
`await request.body()`	the raw body as bytes, `b'{"k":"v"}'`

call_next and the response. `call_next` is a function FastAPI gives your middleware. Calling it

passes the request on, to the next middleware if there is one, and finally to the matching route; it returns the response that came back. That response is an object too: `response.status_code`, `response.headers`. A middleware may change it, or even return a different one, before it goes to the client.

Reading the body in a middleware. `await request.body()` returns the raw body as `bytes`, and `.decode()` turns it into a string. It must be awaited because the body may still be arriving over the network when the middleware starts: the request object is created as soon as the headers are in. (Starlette, which FastAPI is built on, keeps the body after the first read, so the route can still read it.)

Exercises 3.1–3.8

Create `chapter3/phonebook_backend/main.py`. Ids are strings for now.

3.1: Phonebook backend, step 1

`GET /api/persons` returns this hard-coded list:

```
Person(id="1", name="Maren Holt", number="040-123456"),
Person(id="2", name="Idris Okafor", number="39-44-5323523"),
Person(id="3", name="Lena Vasquez", number="12-43-234345"),
Person(id="4", name="Tobias Rhee", number="39-23-6423122"),
```

Run it with `uvicorn main:app --reload --port 3001`.

3.2: Phonebook backend, step 2

`GET /info` returns an HTML page showing how many persons the phonebook has and the time of the request (`datetime.now()`).

3.3: Phonebook backend, step 3

`GET /api/persons/{id}` returns one person, or `404` if there is none.

3.4: Phonebook backend, step 4

`DELETE /api/persons/{id}` removes a person and answers `204`.

3.5: Phonebook backend, step 5

`POST /api/persons` adds a person and answers `201` with it. Generate the id randomly, from a range big enough that collisions are unlikely. The `secrets` module is made for this: `secrets.token_hex(6)` returns 12 random hex characters.

3.6: Phonebook backend, step 6

Reject bad requests:

- a missing or empty name or number: `422`, through the request model's rules;
- a name that already exists: `400` with `{"detail": "name must be unique"}`, through `HTTPException`.

3.7: Phonebook backend, step 7

Add a middleware that logs every request: method, path, status and time taken.

3.8*: Phonebook backend, step 8

For `POST` requests, also log the body.

b. Deploying the app to the internet

Connecting the frontend to your backend

Your notes frontend still talks to the mock server. Point it at FastAPI. The backend's URLs start with `/api`, so in `note_service.py` change:

```
BASE_URL = "http://localhost:3001/notes"
```

to:

```
import os

BACKEND_URL = os.environ.get("BACKEND_URL", "http://localhost:3001")
BASE_URL = f"{BACKEND_URL}/api/notes"
```

Locally nothing needs to be set: the default is your local backend. Once deployed, the frontend gets the backend's internet address through the environment variable `BACKEND_URL`, without any code change. Start both (`uvicorn ...` in one terminal, `reflex run` in another), and the app works as before, now on your own server.

The frontend didn't change otherwise. The mock server and FastAPI speak the same REST conventions, so only the URL changed. That's the payoff of keeping all requests in one service module.

Why there's no CORS problem

The original course has a whole section on **CORS**. Browsers enforce a rule called the *same-origin policy*: JavaScript on a page from one origin (scheme + host + port, such as `http://localhost:3000`) may not read responses from another origin (`http://localhost:3001`) unless that server explicitly allows it with special headers. That's CORS, Cross-Origin Resource Sharing.

In a Reflex app, the requests to your backend are not made by the browser. They're made by *httpx, in your event handlers, on the Reflex server*. The same-origin policy is a browser rule; it doesn't apply to one server talking to another. So FastAPI needs no CORS setup here. If you ever call it from JavaScript in a browser, add FastAPI's `CORSMiddleware`.

Render

What hosting means. So far your servers run on your own computer, at `localhost`. Nobody else can reach them, and they stop when you close the terminal. To be usable by others, a server must run on a computer that is always on, connected to the internet, with a public address. Renting

such a computer and setting it up yourself (installing Python, keeping it updated, restarting the app if it crashes, setting up HTTPS certificates) is a lot of work. A **hosting service** does that part for you: you give it your code and the commands to run it.

Render is a hosting service with a free plan. It runs your app on its computers and gives it an address like `https://notes-backend-xxxx.onrender.com`. Each app you host there is a **web service**.

What Render does on every deploy:

1. It downloads (clones) your repository from GitHub, at the latest commit.
2. It goes into the **root directory** you configured and runs your **build command**, which installs the dependencies.
3. It runs your **start command**, which starts the server, and passes it a port number in the environment variable `PORT`.
4. It points your public address at the new version, and stops the old one. Render's own servers receive the HTTPS traffic from the internet, handle the encryption, and forward each request as plain HTTP to your app's port.

Each backend needs its own `requirements.txt`, because Render installs exactly that file. For the notes backend at this point:

```
fastapi==0.142.2
uvicorn==0.54.0
```

Commit and push. Then, on Render:

1. Sign up with GitHub and create a **New Web Service**. Choose your repository.
2. **Root Directory**: `chapter3/notes_backend`. Render runs every command from there.
3. **Runtime**: Python. **Instance type**: Free.
4. **Build Command**: `pip install -r requirements.txt`.
5. **Start Command**: `uvicorn main:app --host 0.0.0.0 --port $PORT`.
6. Under **Environment**, add `PYTHON_VERSION` with your local version (for example `3.13.16`), so the server runs the same Python as you.

Two parts of the start command are new:

- `--host 0.0.0.0`. By default uvicorn listens on `127.0.0.1`, which accepts connections only from the same computer. `0.0.0.0` means "every network interface", so connections from outside reach it.
- `--port $PORT`. Render chooses the port and passes it in the environment variable `PORT`. The shell replaces `$PORT` with its value.

Every push to GitHub now redeploys the service automatically. The service's dashboard has several tabs:

- **Events**: the list of deploys, each with its commit and its state (in progress, "Deploy live", failed).
- **Logs**: everything your server prints, including your middleware's lines and any traceback. When a deploy fails or the app answers `500`, look here first.

- **Environment**: the environment variables your app receives. Saving a change redeploys the service.
- **Settings**: the root directory, build and start commands you entered when creating the service.

The free plan sleeps. A free service stops after about 15 minutes without requests. The next request wakes it, which takes up to a minute. Data kept in memory is lost on every sleep and every deploy, so for now the notes reset to the three starting ones. Chapter 3c fixes that.

Deploying the Reflex frontend

The frontend is a second web service:

- **Root Directory**: `chapter2/notes`.
- **Build Command**: `pip install -r requirements.txt`, with a `requirements.txt` containing `reflex` and `httpx` with their versions.
- **Start Command**: `reflex run --env prod --single-port --backend-port $PORT`.
- **Environment**: `BACKEND_URL` = your backend's Render address, without a trailing slash; and `PYTHON_VERSION`.

What the start command does:

- `--env prod` builds an optimised, minified version of the JavaScript, instead of the development server with automatic reloading.
- `--single-port` serves the compiled page and the Reflex backend (its WebSocket) from *one* port, because Render gives each service only one. It listens on `0.0.0.0` by itself.
- `--backend-port $PORT` makes that one port Render's.

The page finds its backend by itself: when no API URL is configured, the JavaScript connects to the same address it was loaded from. The first start takes a few minutes, because the production build runs at startup.

Exercises 3.9–3.11

3.9: Phonebook backend, step 9

Make the phonebook frontend use your backend: read `BACKEND_URL` from the environment with a local default, and use the `/api/persons` paths. Everything except changing a number should work (the backend has no update route yet).

3.10: Phonebook backend, step 10

Deploy the phonebook backend to Render. Check `/api/persons` and `/info` at its address.

3.11: Full stack phonebook

Deploy the phonebook frontend to Render, pointing at the deployed backend. Add both addresses to `README.md` at the root of your repository.

c. Saving data to PostgreSQL

Right now your backends forget everything on every restart, deploy or sleep, because the data is a Python list in memory. This section stores it in a **database**, a separate program built to keep data safe and permanent.

Databases, relational databases and SQL

A **database server** is a program whose only job is storing data:

- it writes to disk and survives restarts;
- it handles many clients at once;
- it makes sure two simultaneous changes don't corrupt each other.

Your backend becomes its client, just as the Reflex backend is a client of your FastAPI backend.

PostgreSQL ("Postgres") is a **relational database**. It stores data in **tables**, which work like spreadsheets:

```
table: notes
┌────┬──────────────────────────┬─────────────┐
│ id │ content                  │ important │   ← columns: fixed names and types
├────┼──────────────────────────┼─────────────┤
│  1 │ HTML is easy             │ true      │   ← rows: one per note
│  2 │ Browser can execute only │ false     │
└────┴──────────────────────────┴─────────────┘
```

- Every **column** has a name and a type (`integer`, `text`, `boolean`, ...). The database refuses values of the wrong type.
- Every row has a **primary key**, here `id`: a value that uniquely identifies the row. The database creates it automatically (1, 2, 3, ...) when a row is inserted, using a **sequence**, a counter that only goes up and never hands out the same number twice, even after rows are deleted. That gives you server-generated ids for free.

You talk to the database in **SQL**:

```
SELECT * FROM notes WHERE important = true;
INSERT INTO notes (content, important) VALUES ('new note', false);
UPDATE notes SET important = true WHERE id = 2;
DELETE FROM notes WHERE id = 2;
```

How to read them:

- `SELECT columns FROM table WHERE condition` reads rows. `*` means "every column". Without `WHERE`, every row is returned.
- `INSERT INTO table (columns) VALUES (values)` adds a row. The `id` isn't listed, so the database fills it in.
- `UPDATE table SET column = value WHERE condition` changes the rows that match. Without `WHERE` it would change *every* row.
- `DELETE FROM table WHERE condition` removes the rows that match, again every row without `WHERE`.

Text values are written in single quotes, and every statement ends with `;`.

In this chapter you won't write SQL by hand. **SQLAlchemy** writes it for you from Python code (chapter 13 goes deeper into SQL itself). Still, every SQLAlchemy call below ends up as one of these four kinds of statement, and keeping that in mind makes SQLAlchemy much easier to understand.

A database in the cloud: Neon

Your backend runs on Render, so the database must be reachable from the internet too. We use **Neon**, a hosted PostgreSQL service. Its free plan is permanent and needs no credit card, gives 1 GB of storage per project, and **suspends the database after 5 minutes without use**; it wakes on the next query, similar to Render's sleeping. The original course uses MongoDB Atlas the same way: one hosted database for development and production.

1. Sign up at neon.com, with GitHub or Google.
2. Create a project named `fso-py` and pick the region closest to you.
3. On the project dashboard, click **Connect**. Neon shows a **connection string**, something like:

```
postgresql://neondb_owner:AbC123xyz@ep-cool-name-123456.eu-central-1.aws.neon.tech/neon
↪  db?sslmode=require&channel_binding=require
```

That one line contains everything needed to connect:

```
postgresql://  neondb_owner : AbC123xyz @ ep-cool-name-...neon.tech / neondb ?
↪  sslmode=require&...
└─ scheme ─┘   └── user ──┘  └password┘  └──────────── host ──────────┘ └db──┘ └─ options:
↪  always encrypt ─┘
```

The password is in the string, so the whole string is a secret. It never goes into code or Git.

What you see in Neon's dashboard. A **project** is your space in Neon. Inside it, Neon creates a **branch** called `main`: a complete copy of a PostgreSQL server's data (Neon can make more branches, like Git branches, but you only need `main`). The branch holds a **database** named `neondb` and a **role** (a user account) named `neondb_owner`, which are the database and user in the string above. The left menu has **Tables**, which shows each table's rows like a spreadsheet, and **SQL Editor**, where you can type SQL and run it directly.

Installing the libraries

From the repo root, with the venv active:

```
pip install sqlalchemy "psycopg[binary]" python-dotenv
pip freeze > requirements.txt
```

- **SQLAlchemy** (2.1.3) turns your Python code into SQL, and the results back into Python objects.
- **psycopg** (3.3.6) is the **driver**: the low-level library that actually connects to PostgreSQL and sends SQL to it. SQLAlchemy uses it underneath; you never call it directly. The `[binary]` extra installs a ready-built version, so nothing needs compiling.

- **python-dotenv** (1.2.4) loads environment variables from a file, as explained below.

Why two libraries to reach one database

PostgreSQL is a separate program, usually on another computer. It doesn't understand Python; it understands SQL, sent over a network connection in PostgreSQL's own message format (its **wire protocol**). Getting from your Python code to it takes several layers:

```
your code         note = Note(content="hi"); db.add(note); db.commit()
   |
SQLAlchemy ORM    keeps track of your objects, decides which SQL is needed
   |
SQLAlchemy Core   writes the SQL text:  INSERT INTO notes (content, important)
                  VALUES (%(content)s, %(important)s) RETURNING notes.id
   |
psycopg           opens the connection, sends the SQL and the values, reads the reply
   |  network
PostgreSQL        runs the statement, stores the row, answers with the new id
```

What psycopg does. It's the only part that knows PostgreSQL's wire protocol. With psycopg alone you could already use the database: connect, send an SQL string, get back rows as tuples like `(1, 'HTML is easy', True)`. Every Python program that talks to PostgreSQL uses a driver like this underneath.

What SQLAlchemy adds on top. Working with the driver directly means writing SQL strings by hand, remembering which position in each tuple is which column, and converting rows into objects and back yourself. It also invites a classic security hole, **SQL injection**:

```python
# never do this
name = request_data["name"]                # what the user typed
sql = f"SELECT * FROM persons WHERE name = '{name}'"
```

If someone types `x' OR '1'='1` as their name, the SQL becomes `... WHERE name = 'x' OR '1'='1'`, which matches every row: the user's text has become part of the command. SQLAlchemy never pastes values into the SQL text. It writes named placeholders, such as `%(content)s` above, and sends the values separately, so the database always treats them as data. SQLAlchemy has two layers:

- **Core** builds SQL from Python expressions. It also manages connections. For example, `select(Note).where(Note.id == 5)` becomes this SQL, with the value `5` sent separately:

  ```
  SELECT notes.id, notes.content, notes.important FROM notes WHERE notes.id =
  ↪  %(id_1)s
  ```

- The **ORM** (Object–Relational Mapper) sits on top. You describe each table as a Python class, and work with objects; the ORM decides which SQL statements are needed to make the database match them.

This chapter uses the ORM, which uses Core, which uses psycopg.

Also add them to `chapter3/notes_backend/requirements.txt`, which Render uses:

```
fastapi==0.142.2
uvicorn==0.54.0
```

```
sqlalchemy==2.1.3
psycopg[binary]==3.3.6
python-dotenv==1.2.4
```

Keeping the secret in .env

In 2e you typed `export WEATHER_API_KEY=...` in every new terminal. **python-dotenv** saves that work: you write variables into a file named `.env`, and your code loads them into the environment at startup. Create `chapter3/notes_backend/.env`:

```
DATABASE_URL=postgresql+psycopg://neondb_owner:AbC123xyz@ep-cool-name-123456.eu-central
↪  -1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

Change the start of Neon's string from `postgresql://` to `postgresql+psycopg://`. SQLAlchemy supports several PostgreSQL drivers, and the part after + tells it which one to use. With plain `postgresql://`, it looks for an older driver (psycopg2) that you didn't install.

Check that Git ignores it. Your `.gitignore` should contain `.env`, so `git status` must *not* list the file. Verify this once, now. A committed database password is public as soon as you push, even if you delete it in a later commit, because Git history keeps it.

The new file structure

The backend grows from one file to four, each with one job:

```
chapter3/notes_backend/
├── .env                ← NEW: DATABASE_URL (secret, ignored by Git)
├── requirements.txt
├── main.py             ← the app and its routes
├── database.py         ← NEW: the connection to the database
├── models.py           ← NEW: database tables, as Python classes (SQLAlchemy)
└── schemas.py          ← NEW: shapes of request and response bodies (Pydantic)
```

uvicorn runs from this folder, so the files import each other directly, `from database import ...`, without the leading dot used in Reflex apps.

database.py: connecting

```python
import os

from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

load_dotenv()

DATABASE_URL = os.environ["DATABASE_URL"]

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine)


class Base(DeclarativeBase):
```

```
    pass


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Piece by piece:

- `load_dotenv()` reads `.env` and puts its variables into `os.environ`. On Render there's no `.env` file: you set `DATABASE_URL` in Render's dashboard, and `load_dotenv()` simply finds nothing to load. **A variable that is already set is never overwritten by the file.** Chapter 4b relies on this.
- `create_engine(...)` creates the **engine**, the object that knows how to reach the database. It doesn't connect yet. It keeps a **connection pool**: a few open connections that it reuses, because opening a new database connection is slow (several network round trips, plus setting up encryption).
- `pool_pre_ping=True` matters with Neon. When Neon suspends after 5 minutes, the connections waiting in the pool are cut. Without this option, the next request would take a dead connection from the pool and fail. With it, the engine first sends a tiny test query, and replaces a dead connection with a fresh one.
- `sessionmaker(bind=engine)` returns a *factory*: a class you call to create a new **session** that uses this engine. `SessionLocal()` creates one. A session is your workspace for one piece of work with the database: you load objects through it, change them, add new ones, and **commit** to save everything at once. How it does that is explained in detail after the routes.
- `Base` is the parent class for your table classes (next file). `DeclarativeBase` gives it the machinery that turns a class into a table description. When Python runs the body of a class that inherits from `Base`, that machinery reads the class's `__tablename__` and column declarations and registers a description of the table in a shared catalogue, `Base.metadata`. After `models.py` has been imported, `Base.metadata.tables` contains `{"notes": ...}`. `create_all` (below) reads that catalogue. `pass` just means the class body is empty: everything it needs comes from `DeclarativeBase`.
- `get_db` hands out one session per request and always closes it afterwards. It is a **generator function** (1b): calling it doesn't run it; each time it is resumed it runs until the next `yield`. The code before `yield` creates a session, `yield db` hands the session out and pauses, and when the generator is resumed after the request, `finally` closes the session. Because it's a `try/finally`, the session is closed even if the route raised an exception. How FastAPI drives it is explained below.

Transactions. A commit is the end of a **transaction**: everything since the last commit is saved together, or not at all. If something fails before the commit, nothing is saved, and until the commit, no other request can see your changes. That's how the database guarantees that two simultaneous requests don't leave half-finished data behind.

models.py: the table, as a Python class

```python
from sqlalchemy.orm import Mapped, mapped_column

from database import Base


class Note(Base):
    __tablename__ = "notes"

    id: Mapped[int] = mapped_column(primary_key=True)
    content: Mapped[str]
    important: Mapped[bool] = mapped_column(default=False)
```

This is the **ORM** (Object–Relational Mapper) at the heart of SQLAlchemy: a Python class describes a table, each instance is one row, and each attribute is a column.

- `__tablename__` is the table's name in the database. Name it after what one row is, in the plural: `notes`, `persons`.
- `Mapped[int]` declares a column and its Python type. SQLAlchemy picks the matching database type: `int` → `integer`, `str` → `varchar`, `bool` → `boolean`.
- `mapped_column(primary_key=True)` makes `id` the primary key. For an integer primary key, PostgreSQL generates the values automatically.
- `mapped_column(default=False)` sets a default for rows created without a value.
- A column is **required** (`NOT NULL` in the database) unless its type includes `| None`.

One name, two meanings. After the class is set up, `Note.content` on the *class* and `note.content` on an *object* are different things:

- `note.content`, on an object, is the value of that row's column: a normal `str`.
- `Note.content`, on the class, is a column object (an `InstrumentedAttribute`). Comparing it doesn't give `True` or `False`; it builds a piece of SQL. `Note.content == "hi"` becomes `notes.content = :content_1`, with `"hi"` kept aside as the value. That's how `select(Note).where(Note.content == "hi")` works, in the same way Reflex Vars build JavaScript (1c).

Before an object is saved. `Note(content="hi")` creates a plain Python object. Nothing is sent to the database, and the values the database will fill in don't exist yet: `note.id` is `None`, and so is `note.important`, because `default=False` is applied when the row is inserted, not when the object is created.

Two kinds of models now exist, and they're not the same thing:

	SQLAlchemy model (`models.py`)	**Pydantic schema** (`schemas.py`)
Describes	a **table** in the database	the **JSON** in a request or response
Used for	reading and writing rows	validating input, shaping output
Example	`Note`	`NoteCreate`, `NoteOut`

They often have similar fields, but they answer different questions: "what's stored?" versus "what may a client send, and what do we send back?". Keeping them apart means you can, for example, store a password hash in the table but never include it in any response schema (chapter 4).

schemas.py: request and response shapes

```python
from pydantic import BaseModel, ConfigDict, Field


class NoteCreate(BaseModel):
    content: str = Field(min_length=1)
    important: bool = False


class NoteUpdate(BaseModel):
    content: str | None = Field(default=None, min_length=1)
    important: bool | None = None


class NoteOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    content: str
    important: bool
```

- `NoteCreate` is the same as in 3a.
- `NoteUpdate` is for `PATCH`. A field is optional only if it has a default value, so every field here has `= None`: a PATCH sends only the fields that change. `min_length=1` still applies when `content` *is* sent.
- `NoteOut` is what responses contain.

What ConfigDict is. `Field(...)` sets rules for *one field*. `ConfigDict` is a set of **switches for the whole model**. Each switch has a default, and you list only the ones you want to turn on. It goes in a class attribute that must be named exactly `model_config`, because that's the name Pydantic looks for. `ConfigDict` itself is just a dict: `ConfigDict(from_attributes=True)` and `{"from_attributes": True}` do the same, and the `ConfigDict(...)` form only adds spell-checking in Pylance.

The switch from_attributes. By default, Pydantic builds a model from a **dict**, reading values as `data["id"]`. Your routes will have SQLAlchemy objects instead, whose values are **attributes**: `note.id`, `note.content`. With `from_attributes=True`, Pydantic can also read attributes:

```python
class A(BaseModel):
    id: int

class B(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int

note = db.get(Note, 1)      # a database row, read as note.id

A.model_validate(note)      # error: "Input should be a valid dictionary"
B.model_validate(note)      # works: B also reads note.id
```

model_validate builds a model from existing data: it reads the values, checks them against the model's fields and types, and returns a model object. Pydantic calls this validating, which is where the name comes from. `NoteOut.model_validate(note)` turns a database row into the re-

sponse shape.

The id is now an int. In 3a, ids were strings because the mock server used strings and the frontends expected them. Now the database creates them, and PostgreSQL's ids are integers. The JSON now contains `"id": 2` instead of `"id": "2"`, so the frontends must change too (below).

main.py: the routes, using the database

```
from fastapi import Depends, FastAPI, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from database import Base, engine, get_db
from models import Note
from schemas import NoteCreate, NoteOut, NoteUpdate

Base.metadata.create_all(engine)

app = FastAPI()


@app.get("/api/notes")
def get_notes(db: Session = Depends(get_db)) -> list[NoteOut]:
    notes = db.scalars(select(Note)).all()
    return [NoteOut.model_validate(n) for n in notes]


@app.get("/api/notes/{note_id}")
def get_note(note_id: int, db: Session = Depends(get_db)) -> NoteOut:
    note = db.get(Note, note_id)
    if note is None:
        raise HTTPException(status_code=404, detail="note not found")
    return NoteOut.model_validate(note)


@app.post("/api/notes", status_code=201)
def create_note(data: NoteCreate, db: Session = Depends(get_db)) -> NoteOut:
    note = Note(content=data.content, important=data.important)
    db.add(note)
    db.commit()
    db.refresh(note)
    return NoteOut.model_validate(note)


@app.patch("/api/notes/{note_id}")
def update_note(note_id: int, data: NoteUpdate, db: Session = Depends(get_db)) ->
    NoteOut:
    note = db.get(Note, note_id)
    if note is None:
        raise HTTPException(status_code=404, detail="note not found")
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(note, field, value)
    db.commit()
    db.refresh(note)
    return NoteOut.model_validate(note)
```

```python
@app.delete("/api/notes/{note_id}", status_code=204)
def delete_note(note_id: int, db: Session = Depends(get_db)) -> None:
    note = db.get(Note, note_id)
    if note is not None:
        db.delete(note)
        db.commit()
```

Keep your logging middleware and the HTML root route from 3a too. They're left out here only to keep the example short.

`Base.metadata.create_all(engine)` runs once, when the server starts. It looks at every class that inherits from `Base` and creates the missing tables (`CREATE TABLE notes ...`). It needs `models` to have been imported first, which is why it comes after the imports: a class SQLAlchemy has never seen can't be created. Tables that already exist are left alone, **even if you changed the class**: adding a column to `Note` later won't add it to the existing table. Changing existing tables is called a **migration**, done with a tool called Alembic later in the course. Until then, if you change a model, delete the table in Neon's dashboard and let `create_all` recreate it.

`db: Session = Depends(get_db)` is FastAPI's **dependency injection**. It reads as: "this route needs a `db`; get it by calling `get_db`". For every request, FastAPI:

1. calls `get_db`, which runs up to `yield` and creates a session;
2. passes that session to your route as `db`;
3. after the response is sent, resumes `get_db` after `yield`, so `finally` closes the session.

Every route gets its own fresh session and never has to open or close one. `Depends` is a general tool: in chapter 4 the same mechanism provides "the currently logged-in user" to routes.

What `db: Session` means. `Session` is the class of the object `get_db` yields, imported from `sqlalchemy.orm`. As a type hint it does two things: Pylance knows what `db` is and can check your calls on it, and the default `= Depends(get_db)` tells FastAPI (rule 3 in "Where FastAPI gets each argument from") that this argument comes from a dependency, not from the request.

Reading:

- `select(Note)` builds the query `SELECT * FROM notes`. Building it doesn't run it.
- `db.scalars(...)` runs it, and `.all()` returns a list of `Note` objects, one per row. (Without `scalars`, `db.execute` returns rows as tuples; `scalars` takes the first item of each, here the `Note`.)
- `db.get(Note, note_id)` loads one row by primary key, or returns `None` if there isn't one. It's the database version of your `next(...)` search.

Writing:

- `db.add(note)` tells the session about a new object. Nothing is sent to the database yet.
- `db.commit()` sends every pending change (`INSERT`, `UPDATE`, `DELETE`) and makes it permanent.
- `db.refresh(note)` reloads the object's values from the database right away. In these routes it isn't strictly needed: after a commit, the object reloads itself the next time you read it. The next section explains exactly when `refresh` matters.
- Changing a loaded object's attribute (`setattr(note, field, value)`) is enough for SQLAlchemy to know an `UPDATE` is needed at the next commit. `setattr(obj, "content",`

`"x")` is the same as `obj.content = "x"`, with the attribute's name given as a string.

- `db.delete(note)` marks the row for deletion at the next commit.

`data.model_dump(exclude_unset=True)` turns the Pydantic object into a dict containing **only the fields the client actually sent**. For `PATCH {"important": true}` that's `{"important": True}`: `content` isn't included, so it isn't overwritten. Without `exclude_unset`, every missing field would come back as its default, `None`.

note_id: int in the path. FastAPI now converts the id to an integer, so `/api/notes/abc` is rejected with `422` before your function runs. The original course needs special error handling for a "malformatted id" at this point; FastAPI's validation handles it for you.

Does the route need model_validate? Strictly, no. With `return note`, FastAPI sees the return type `-> NoteOut` and does the same conversion itself, using `from_attributes`. Writing `model_validate` is still the better habit, for two reasons. First, the code says what it does: the function promises a `NoteOut`, and `return note` would actually return a `Note`, a mismatch Pylance flags. Second, you see where the filtering happens: only the fields `NoteOut` declares leave the server. In chapter 4 that's what keeps password hashes private.

What the routes do, tested against a real PostgreSQL database:

Request	Response
`GET /api/notes` on an empty database	`200 []`
`POST {"content": "HTML is easy", "important": true}`	`201 {"id": 1, ...}`
`GET /api/notes/abc`	`422`
`GET /api/notes/999`	`404`
`PATCH /api/notes/2 {"important": true}`	`200`, only `important` changed
`PATCH /api/notes/2 {"content": ""}`	`422`
`DELETE /api/notes/2`, twice	`204`, both times

Inside the session

The routes above work, but several things happen that the code doesn't show: `add` sends nothing, changing an attribute is enough for an `UPDATE`, and the new `id` appears after a commit. All of it is the **session** at work. These are the real queries SQLAlchemy sent, recorded step by step.

1. The session keeps a list of pending work (the unit of work).

```
note = Note(content="hi")
db.add(note)        # no query: the session only notes "insert this later"
db.commit()         # INSERT INTO notes (content, important) VALUES (...) RETURNING
    notes.id
```

A session keeps track of three groups of objects: **new** ones (added, not yet in the database), **dirty** ones (loaded, then changed), and **deleted** ones. Sending the SQL for all of them is called a **flush**. `commit()` first flushes, then tells the database to make the transaction permanent. (You can also call `db.flush()` yourself: the SQL is sent, so the new row gets its id, but nothing is permanent until the commit.)

2. Changing an attribute marks the object as dirty.

```
note = db.get(Note, 1)
```

```
note.content = "changed"    # no query; the session now lists note as dirty
db.commit()                 # UPDATE notes SET content=... WHERE notes.id = ...
```

Every object the session loaded is **watched**: the mapped attributes record each change. At the flush, SQLAlchemy writes an `UPDATE` containing only the columns that changed. That's why the PATCH route needs nothing more than `setattr` and `commit`.

3. One object per row (the identity map).

```
a = db.get(Note, 1)
b = db.get(Note, 1)
a is b                      # True, and the second get sent no query at all
```

The session remembers every object it has loaded, by table and primary key. Asking again for the same row returns the *same* Python object, so two parts of your code can't end up holding two different, conflicting copies of one row. `db.get` even skips the query when the object is already known. (A `select(...)` always runs its query, but if a row it returns is already known, you still get the existing object.) The map belongs to one session: another session loads its own, separate objects.

4. After a commit, objects are expired and reload themselves.

```
db.commit()
note.id             # SELECT notes.id, notes.content, notes.important FROM notes WHERE
    ...
```

Once a transaction ends, other requests may change the rows at any moment, so the values in your objects could be stale. So by default `commit()` marks every object as **expired**: its values are discarded, and the next time you read any attribute, the session silently runs a `SELECT` and refills it. That's how `note.id` has the new id after a commit: the reload reads what the database created.

5. What refresh does, and when it matters. `db.refresh(note)` does that reload *immediately*, instead of at the next read. Inside a route, where the session is still open when `model_validate` reads the values, there's no difference: the values would be reloaded on read anyway. It matters when the object is used *after its session is closed*:

```
with SessionLocal() as db:
    note = Note(content="hi")
    db.add(note)
    db.commit()         # note is now expired
print(note.id)          # DetachedInstanceError: there's no session left to reload it
```

Once its session is closed, an object is **detached**: it still exists in Python, but it can't load anything. With `db.refresh(note)` before the end of the `with` block, the values are loaded while the session is open, and `print(note.id)` works. The routes in this book keep `refresh` as a clear, harmless statement of "I want the database's values now".

6. Rollback and close. If something goes wrong before a commit, `db.rollback()` cancels the transaction: everything sent since the last commit is undone in the database, and the session forgets its pending changes. `db.close()` (which `get_db` calls for you) rolls back anything uncommitted, gives the connection back to the engine's pool, and detaches all its objects. That's why each request uses its own session: what one request does can never leak into another's half-finished work.

From SQL to SQLAlchemy

The SQL you know, and its SQLAlchemy 2.x equivalent:

SQL	SQLAlchemy
`SELECT * FROM persons`	`db.scalars(select(Person)).all()` → a list
`... WHERE name = 'X'`	`db.scalars(select(Person).where(Person.name == "X")).all()`
first match only	`db.scalar(select(Person).where(...))` → one object or `None`
`... WHERE id = 3`	`db.get(Person, 3)` → one object or `None`
`SELECT count(*) FROM persons`	`db.scalar(select(func.count()).select_from(Person))`
`ORDER BY name`	`select(Person).order_by(Person.name)`
`INSERT`	`db.add(obj)` then `db.commit()`
`UPDATE`	change attributes, then `db.commit()`
`DELETE`	`db.delete(obj)` then `db.commit()`

The pattern: `select(...)` builds the query, then `db.scalars(...)` runs it for many rows and `db.scalar(...)` for one value.

- `.where(Person.name == "X")` becomes `WHERE name = 'X'`. `Person.name == "X"` doesn't compare anything in Python; like a Reflex Var, the column object builds an expression that SQLAlchemy turns into SQL.
- `func.<name>` calls any SQL function: `func.count()`, `func.lower(Person.name)`. Import it with `from sqlalchemy import func`.
- `select(func.count()).select_from(Person)` counts rows *in the database*, without loading them into Python. Loading every row just to call `len()` on the list would get slower as the table grows.

The most useful page of the documentation is SQLAlchemy's "ORM Querying Guide", section "SELECT statements".

Seeing the data

Run the backend as before (`uvicorn main:app --reload --port 3001`), create a few notes in `/docs`, then restart the server. **The notes are still there.**

You can look at the table directly in Neon's dashboard. **Tables** shows the rows, and the **SQL Editor** runs SQL by hand. Try `SELECT * FROM notes;` there. Then add a note in the SQL Editor and reload `/api/notes`:

```
INSERT INTO notes (content, important) VALUES ('from SQL', false);
```

The note appears, because the backend no longer holds data of its own; it just asks the database.

Error handling in one place

Errors can now happen in places your code doesn't control: the database might be unreachable, or Neon might fail to wake up. Instead of wrapping every route in `try/except`, FastAPI lets you

register an **exception handler**: a function that runs whenever a given type of exception escapes *any* route. In `main.py`:

```
from fastapi.responses import JSONResponse
from sqlalchemy.exc import OperationalError


@app.exception_handler(OperationalError)
def database_unavailable(request, exc):
    print("database error:", exc.orig)
    return JSONResponse(status_code=503, content={"detail": "database unavailable"})
```

- `OperationalError` is what SQLAlchemy raises when the database can't be reached or the connection breaks. `exc.orig` is the driver's original error, with the real reason.
- Without this handler, the client gets a generic `500 Internal Server Error`, and the traceback appears in the server log.
- With it, the client gets **503 Service Unavailable** ("try again later") with a readable message, and the log still records the real reason.
- `JSONResponse` builds a response by hand, with any status code and body.

This is FastAPI's version of the original course's error-handling middleware: error responses decided in one place for the whole app.

Seeing the 503 yourself. You have to break the connection while the server can still start. There's a catch: `create_all` connects as soon as the server starts, so with a broken URL the server crashes at startup, before any request reaches your handler.

1. Comment out `Base.metadata.create_all(engine)`. That's safe, because the table already exists.
2. In `.env`, point the URL at a port where nothing listens, such as `localhost:5999`. You get "connection refused" immediately, instead of waiting for a failed name lookup.
3. Restart uvicorn: `--reload` only watches `.py` files, not `.env`.
4. Open `/api/notes`: `503` and `{"detail": "database unavailable"}`, with `database error: ...` in the terminal.
5. Undo both changes and restart.

Updating the notes frontend

The ids are integers now:

- In `chapter2/notes/notes/models.py`, change `id: str` to `id: int`.
- Every handler or service function that takes an id changes too: `toggle_importance(self, note_id: int)`, `update_important(note_id: int, ...)`.

The frontend change matters even though Python wouldn't complain if you skipped it: `Note(**n)` would accept the integer while the hints still said `str`, and Pylance's checks would be based on a lie. Toggling importance now works end to end, because the backend has a `PATCH` route.

Deploying

1. Commit and push. `git status` must *not* show `.env`.
2. In Render, open the notes backend → **Environment** → add `DATABASE_URL` with the same value as in your `.env` (the one starting `postgresql+psycopg://`).
3. Saving the variable triggers a redeploy. When it's live, `/api/notes` on Render shows the **same notes** as locally, since both use the same Neon database.

Data now survives Render's restarts and sleeping.

Working without a network: scripts

Exercise 3.12 is a small command-line script that uses the same `database.py` and `models.py`. Two things are new outside of FastAPI:

```
import sys

from database import SessionLocal

print(sys.argv)     # python cli.py "Noor Haddad" 050-1234567  →  ['cli.py', 'Noor
    Haddad', '050-1234567']

with SessionLocal() as db:
    ...             # use db here; it is closed when the block ends
```

- `sys.argv` is the list of words typed after `python`. `sys.argv[0]` is the script's name. The terminal splits words at spaces, so a name with a space needs quotes.
- A script has no `Depends`. `with SessionLocal() as db:` opens a session and closes it at the end of the block, like the `with` blocks in chapter 2.
- Import `engine`, `SessionLocal` and `Base` from `database`; don't build a second engine. Importing `database` also runs its `load_dotenv()`.

Exercises 3.12–3.18

Convert the phonebook backend to PostgreSQL. Use a separate table, `persons`, in the same Neon database.

```
chapter3/phonebook_backend/
├── .env                ← DATABASE_URL, the same value as the notes backend
├── requirements.txt    ← add the three new packages
├── main.py
├── database.py
├── models.py
├── schemas.py
└── cli.py              ← exercise 3.12 only
```

Limits for these exercises:

- SQLAlchemy ORM only, no SQL strings in Python code.
- Each route gets its session through `Depends(get_db)`.

- Ids are integers: in the backend's models, schemas and path parameters, and in the frontend's `Person.id` and every handler and service function that takes an id.
- Responses use a Pydantic schema built with `model_validate`.

3.12: Command-line database

Write `cli.py`, which works with the `persons` table from the terminal. With no arguments it prints every entry:

```
$ python cli.py
phonebook:
Maren Holt 040-123456
Idris Okafor 39-44-5323523
```

With a name and a number it adds an entry:

```
$ python cli.py "Noor Haddad" 050-1234567
added Noor Haddad number 050-1234567 to phonebook
```

The script must create the table if it's missing, as `main.py` does.

3.13: Phonebook database, step 1

`GET /api/persons` returns the entries from the database. Check it in the browser and in your deployed phonebook frontend.

3.14: Phonebook database, step 2

New entries from `POST /api/persons` are saved to the database. The duplicate-name check from 3.6 still works, now asking the database (`.where(...)` with `db.scalar`).

3.15: Phonebook database, step 3

`DELETE /api/persons/{id}` removes the entry from the database. Check that the frontend still updates correctly.

3.16: Phonebook database, step 4

Add the central exception handler for database errors (`OperationalError` → `503`), and see it work as described above.

3.17*: Phonebook database, step 5

Add `PATCH /api/persons/{id}`, which updates an entry's number. Your frontend already sends exactly this request when you confirm "replace the old number with a new one?" (exercise 2.15), so that feature now works end to end. A missing id returns `404`, and an empty number is rejected with `422`. The request body has one field, `number`, and it is required.

3.18*: Phonebook database, step 6

`/info` and `GET /api/persons/{id}` read from the database too. For `/info`, count the rows in the database, without loading every person into Python.
Deploy when you're done: `DATABASE_URL` goes into the phonebook backend's Render environment too.

Running PostgreSQL locally

You'll run PostgreSQL on your own machine in three places: in chapter 4 for tests (in Docker), in chapter 12 with Docker Compose, and in chapter 13 with `psql` and Alembic migrations. Your code won't change for any of this: a local database is just a different `DATABASE_URL`, such as `postgresql+psycopg://postgres:secret@localhost:5432/notes`. SQLAlchemy doesn't care where the server runs.

d. Validation and Ruff

Two topics:

1. **Validation:** stopping bad data before it reaches the database, and showing the reason to the user in the frontend.
2. **Ruff:** a tool that finds mistakes in your code and formats it automatically.

We work in `chapter3/notes_backend` and `chapter2/notes` first. The exercises then do the same for the phonebook.

Where validation happens, and why on the server

Validation means checking that incoming data follows your rules before using it. For example: "a note's content must be at least 5 characters".

Your frontend may check a few things, but **the frontend's checks don't protect the backend**. Your FastAPI backend has a public URL, so anyone can send requests to it directly, without your frontend; `/docs` does exactly that. So the backend must check everything itself. Frontend checks only make the app nicer to use, because the user gets feedback without waiting for the server.

A reminder of how FastAPI validates. When a route has a parameter typed with a Pydantic model, such as `data: NoteCreate`:

1. FastAPI reads the JSON body of the request.
2. It tries to build a `NoteCreate` from it.
3. If any rule fails, your function never runs. FastAPI answers **422 Unprocessable Content**: the JSON was readable, but its contents break the rules.

All validation rules therefore go in `schemas.py`.

Rule 1: length limits

This changes existing code. In `schemas.py`, the `content` field of `NoteCreate` currently has `Field(min_length=1)`. Change it to:

```
content: str = Field(min_length=5, max_length=200)
```

- `min_length=5` rejects strings shorter than 5 characters.
- `max_length=200` rejects longer ones. Without a maximum, someone could send a 50 MB note and you'd store it.

Try `POST /api/notes` in `/docs` with `{"content": "abc"}`. The response is `422`, with this body:

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": ["body", "content"],
      "msg": "String should have at least 5 characters",
      "input": "abc",
      "ctx": {"min_length": 5}
    }
  ]
}
```

- `detail` is a **list**: one entry per broken rule, since several fields can fail at once.
- `loc` says where the problem is: `["body", "content"]` means the field `content` in the request body.
- `msg` is a human-readable explanation. The frontend will show it later in this section.

Rule 2: the same rules when updating

In the original course this is a classic bug: rules checked on create are not checked on update, so a PATCH can sneak in invalid data. In FastAPI, a PATCH body is validated by **its own model**, `NoteUpdate`, so it has to carry the same rules.

This changes existing code. In `NoteUpdate`, change `content` to:

```
content: str | None = Field(default=None, min_length=5, max_length=200)
```

- `default=None` makes the field optional: a PATCH may leave `content` out, for example when only toggling `important`.
- `| None` keeps the type honest. Writing `content: str = Field(default=None, ...)` would run (Pydantic doesn't check default values), but Pylance warns "None is not assignable to str", and it would mislead anyone reading the code.

The null problem. `| None` also means a client may *send* `{"content": null}`. Your PATCH route copies the sent fields into the database row, and `None` can't be stored in a `NOT NULL` column: the server would crash with a `500`. The fix belongs in the route.

This changes existing code. In `main.py`, in `update_note`, add `exclude_none=True` to the existing `model_dump` call:

```
for field, value in data.model_dump(exclude_unset=True, exclude_none=True).items():
```

- `exclude_unset=True` drops fields the client didn't send.
- `exclude_none=True` also drops fields the client sent as `null`.

PATCH body	What gets applied
`{"content": null}`	nothing
`{"important": true}`	`{"important": True}`
`{"content": "hello world", "important": null}`	`{"content": "hello world"}`

The problem only appears when a field is both **optional** (it has a default) and could be `None`. A required field with no `None` in its type, such as the phonebook's `number: str = Field(...)`, rejects `null` with 422 by itself.

Rule 3: ignoring surrounding spaces

`"  "` (five spaces) passes `min_length=5`, and so does `" a "`. Neither is a real note. Pydantic can remove spaces at the start and end of every string **before** checking the rules.

This changes existing code. Add a `model_config` line as the first line inside both `NoteCreate` and `NoteUpdate`:

```
class NoteCreate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)

    content: str = Field(min_length=5, max_length=200)
    important: bool = False
```

This is the second `ConfigDict` switch in the book (the first was `from_attributes` in `NoteOut`). Compare:

```
class A(BaseModel):
    content: str

class B(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)
    content: str

A(content="  hi  ").content   # '  hi  '   switch off: the spaces stay
B(content="  hi  ").content   # 'hi'       switch on: the spaces are removed first
```

So `" a "` is checked as `"a"` and rejected as too short, and `" hello world "` is accepted and stored as `"hello world"`.

Model	Switch	Why
`NoteCreate`, `NoteUpdate`	`str_strip_whitespace=True`	clean the client's input before validating it
`NoteOut`	`from_attributes=True`	build the response from a SQLAlchemy object

Rule 4: custom rules with field_validator

Some rules can't be written as `Field(...)` options. For those, you write a Python function and mark it as a **validator** for a field. Example: refuse notes written entirely in capital letters.

This adds code. Add `field_validator` to the existing `from pydantic import ...` line, then add this method inside `NoteCreate`, after `important`:

```
from pydantic import BaseModel, ConfigDict, Field, field_validator


class NoteCreate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)

    content: str = Field(min_length=5, max_length=200)
    important: bool = False

    @field_validator("content")
    @classmethod
    def not_shouting(cls, value: str) -> str:
        if value.isupper():
            raise ValueError("content must not be written in all capital letters")
        return value
```

- `@field_validator("content")` is a decorator that tells Pydantic: "run this function on the `content` field".

- `@classmethod` is required by Pydantic for validators. A normal method receives the object as `self`; a class method receives the *class* as its first argument, named `cls` by convention. Here the object doesn't exist yet, because its data is still being checked, so there's no `self` to give. You won't use `cls`; it just has to be there. Decorators are applied from the bottom up, so `@classmethod` wraps the function first.

- `value` is the field's value. It arrives **after** the `Field` rules and the stripping, so it's already a stripped string of 5–200 characters.

- **Rejecting:** `raise ValueError("...")`. Pydantic catches it and turns it into a `422`, with `msg` set to `"Value error, content must not be written in all capital letters"`. Pydantic adds the `"Value error, "` prefix itself.

- **Accepting:** `return value`. Whatever you return is what gets stored, so a validator can also clean a value, not only reject it.

For the same rule on PATCH, copy the validator into `NoteUpdate`. There `value` can be `None`, so type it `value: str | None` and start with `if value is None: return value`.

Rule 5: format rules with pattern

Format rules are written as a **regular expression** (regex): a small language for describing what a string looks like. `Field(pattern=...)` rejects any string that doesn't match.

Piece	Means
`\d`	one digit (0–9)
`[0-9]`	one digit from 0 to 9, ASCII only
`\d{3}`	exactly 3 digits
`\d{2,4}`	2 to 4 digits
`\d+`	one or more digits
`-`	a literal dash
`^`	the start of the string
`$`	the end of the string

Examples: a 5-digit postal code is `^\d{5}$`; a date like `2026-10-13` is `^\d{4}-\d{2}-\d{2}$`. In code:

```
postal_code: str = Field(pattern=r"^\d{5}$")
```

- **Always use `^` and `$`.** Pydantic accepts a string if the pattern is found *anywhere* in it. Without the anchors, `\d{5}` would accept `"abc12345xyz"`.
- **Write the regex as `r"..."`, a raw string.** In a normal Python string, `\` starts special characters, like `\n` for a new line. In a raw string, `\d` stays exactly `\d`, which is what the regex needs.
- `\d` matches digits from *every* script, including Arabic-Indic ones. Use `[0-9]` when only ASCII digits are allowed.

A failed pattern gives a `422` with `"msg": "String should match pattern '...'"`. That message is accurate, but users don't read regex. A `field_validator` that raises a `ValueError` with your own wording is friendlier.

To experiment, use Python's `re` module in the shell:

```
>>> import re
>>> bool(re.fullmatch(r"\d{5}", "12345"))
True
>>> bool(re.fullmatch(r"\d{5}", "1234"))
False
```

`re.fullmatch` requires the *whole* string to match, so you don't need `^` and `$` there. It returns a match object or `None`, which works directly in an `if`.

Showing backend errors in the frontend

Right now, if the backend rejects a new note, your Reflex app crashes in that handler: `note_service.create` calls `response.raise_for_status()`, which turns the `422` into an `httpx.HTTPStatusError`, and `add_note` doesn't catch it. The fix has two parts: read the error message out of the response, and show it in the error box from 2e.

Part 1: reading the message. The error body has two shapes:

- `422` from validation: `detail` is a **list** of errors, as in Rule 1.
- `400` or `404` from your own `HTTPException(...)`: `detail` is a plain **string**, such as `"name must be unique"`.

The service is the part of the frontend that knows about HTTP, so the conversion goes there. **This adds code** at the end of `chapter2/notes/notes/services/note_service.py`:

```
def error_detail(error: httpx.HTTPStatusError) -> str:
    """Turn the error body sent by the backend into one readable message"""
    detail = error.response.json()["detail"]
    if isinstance(detail, str):
        return detail
    return "; ".join(e["msg"].removeprefix("Value error, ") for e in detail)
```

- `error.response` is the response that caused the exception, and `.json()` parses its body.
- `isinstance(detail, str)` checks whether `detail` is a string. If it is, it's the `400`/`404` shape and can be returned as is.
- Otherwise it's the `422` list. The generator expression takes each entry's `msg` and removes Pydantic's prefix (`str.removeprefix` returns the string without that start, or unchanged if it doesn't start with it), and `"; ".join(...)` combines the messages into one string.

Part 2: showing it. This changes existing code: `add_note` in `chapter2/notes/notes/notes.py` becomes:

```
@rx.event
async def add_note(self, form_data: dict[str, typing.Any]):
    try:
        created = await note_service.create(content=self.new_note, important=random() <
    0.5)
    except httpx.HTTPStatusError as error:
        if error.response.status_code in (400, 422):
            self.error_message = note_service.error_detail(error)
            return NoteState.clear_error_later
        raise
    self.notes.append(created)
    self.new_note = ""
```

This is the same pattern as `toggle_importance`. For the statuses we understand, the message goes into `error_message`, and the background event clears it after 5 seconds. Any other status is re-raised, so real bugs still show up in the terminal. The final `raise` matters for another reason too: without it, a `500` would fall through to `self.notes.append(created)` while `created` was never assigned, and crash with a confusing `UnboundLocalError`. `self.new_note = ""` only runs on success, so after an error the user's text stays in the input and they can fix it.

Try it: run the backend and the frontend, then save `"abc"`, then `"HELLO WORLD"`. Each time, a red box shows the backend's own message.

Ruff: linting and formatting

A **linter** reads your code without running it and reports likely mistakes: unused variables, imports you never use, code that can never run. A **formatter** rewrites your code's layout (spacing, line breaks, quotes) into one consistent style. **Ruff** does both. It replaces ESLint from the original course.

Installing it as a development tool. Ruff is needed only while you write code, never by the running apps. Tools like this are **development dependencies**. They go in a separate file at the

repo root, `requirements-dev.txt`, and **never** in the `requirements.txt` files Render uses, which would install them for no reason. Create `fso-py/requirements-dev.txt`:

```
ruff==0.17.1
```

and install it from the repo root: `pip install -r requirements-dev.txt`.

Configuring it. Ruff reads its settings from `ruff.toml`. It searches the folder of the file being checked, then each parent folder, so one file at the repo root covers every app. Create `fso-py/ruff.toml`:

```
line-length = 100

[lint]
select = ["E4", "E7", "E9", "F", "B"]
ignore = ["F541"]

[lint.flake8-bugbear]
extend-immutable-calls = ["fastapi.Depends"]

[format]
indent-style = "tab"
```

- `line-length = 100`: the formatter wraps lines longer than 100 characters. The default is 88.
- `select` chooses which rule groups are on. Each rule has a code made of letters for the group and a number:
  - `E4`, `E7`, `E9`: basic errors (bad comparisons like `== None`, statements Python can't parse);
  - `F`: real mistakes, such as unused imports, unused variables and undefined names;
  - `B`: "bugbear", common bug patterns, such as a `try/except` that silently hides every error.
- `ignore = ["F541"]` turns off the rule against f-strings without placeholders, since you sometimes write those on purpose.
- `[lint.flake8-bugbear]`: rule `B008` warns against calling a function in a default argument, as in `db: Session = Depends(get_db)`. In normal Python that's a real trap, because the call happens *once*, when the function is defined, not on each call. FastAPI's `Depends(...)` is designed to be used exactly this way (it only records "call `get_db` per request"), so this setting tells Ruff that `Depends` is safe.
- `indent-style = "tab"`: the formatter keeps your tabs instead of converting them to spaces.

Ruff skips everything in `.gitignore` (`.venv`, `.web`, `__pycache__`), so it never checks generated code.

Running it, from the repo root:

```
ruff check .              # report problems; changes nothing
ruff check . --fix        # also fix the problems that are safe to fix automatically
ruff format --check .     # list files the formatter would change; changes nothing
ruff format .             # reformat every file
```

A typical first report:

```
chapter2/phonebook/phonebook/state.py:93:4: F841 Local variable `response` is assigned
↪  to but never used
```

The format is *file:line:column: code message*. Look at the reported line before fixing: an unused variable often means the code meant to do something with it and doesn't.

In VS Code, install the **Ruff** extension (publisher: Astral). It shows problems as you type, using the same `ruff.toml`. To format on save, add to `.vscode/settings.json`:

```
{
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true
  }
}
```

Summary of what changed in the notes example

File	Change
`notes_backend/schemas.py`	`str_strip_whitespace, min_length=5, max_length=200` in both input schemas; the `not_shouting` validator in `NoteCreate`
`notes_backend/main.py`	`exclude_none=True` in `update_note`
`notes/services/note_service.py`	new `error_detail` function
`notes/notes.py`	`add_note` catches `400`/`422` and shows the message
`requirements-dev.txt`	new: `ruff==0.17.1`
`ruff.toml`	new: Ruff configuration

Exercises 3.19–3.22

Limits for these exercises:

- Every validation rule lives in the Pydantic schemas in `schemas.py`. No `if` checks for length or format inside routes.
- Create (`PersonCreate`) and update (`PersonUpdate`) must enforce the same rules for the fields they share.
- The frontend shows the backend's message. Don't copy the rules into the frontend.
- Ruff goes only in `requirements-dev.txt`, never in an app's `requirements.txt`.

3.19: Phonebook database, step 7

In `chapter3/phonebook_backend/schemas.py`:

- a name must be at least 3 characters long, after removing surrounding spaces;
- a number must not be empty, after removing surrounding spaces.

In the phonebook frontend, when adding a person fails with `400` or `422`, show the backend's message in your failure notification (the `"failure"` class from 2.17) instead of crashing. Add an `error_detail` function to `services/person_service.py`, as in the notes example. Check it by adding a person named `Al`.

3.20*: Phonebook database, step 8

A phone number must be at least 8 characters long, and consist of two parts separated by `-`: the first part has 2 or 3 digits, and the second part has only digits.

Valid	Invalid
`09-1234556`	`1234556` (no dash)
`040-22334455`	`1-22334455` (first part too short)
`12-345678`	`10-22-334455` (two dashes)
	`12-3456` (only 7 characters)

Note that both rules are needed: the format alone would accept `12-3`, which is too short. Use `pattern` or a `field_validator`. The rule applies to `PersonUpdate` too, so replacing a number with an invalid one also shows the error in the frontend; that error comes from the handler that updates a number, which so far only handles `404`.

3.21: Deploying the validated version

Commit, push, and wait until both Render services show "Deploy live" in their Events tab. Then check in the *deployed* frontend that the errors from 3.19 and 3.20 appear.

3.22: Lint configuration

Set up Ruff as described above (`requirements-dev.txt`, `ruff.toml`, and the VS Code extension if you use VS Code). Then:

1. Run `ruff check .` and fix everything it reports. Also check every `except` branch in the phonebook frontend: for statuses you don't handle, the error should be raised, not ignored.
2. Run `ruff format .` and commit the result in a **separate commit**, so the formatting changes don't get mixed up with real changes in Git history.
3. `ruff check .` must report `All checks passed!`.

Chapter 4

Testing FastAPI apps and user administration

Sections: a. Project structure and intro to testing · b. Testing the backend · c. User administration · d. Token authentication

This edition covers sections a and b.

a. Project structure and intro to testing

Why split the backend into more files

Your notes backend's `main.py` holds three different kinds of code: the routes, the logging middleware, and the database error handler. As the app grows (users and login come later in this chapter), one file becomes hard to navigate. The fix is the same idea as in 3c, when you split out `database.py`, `models.py` and `schemas.py`: **one job per file**.

The new structure of `chapter3/notes_backend`:

```
notes_backend/
├── .env
├── requirements.txt
├── pytest.ini          ← NEW: pytest settings
├── main.py             ← CHANGED: only creates the app and wires the parts together
├── database.py         (unchanged)
├── models.py           (unchanged)
├── schemas.py          (unchanged)
├── middleware.py       ← NEW: log_requests and database_unavailable, moved from main.py
├── routers/
│   ├── __init__.py     ← NEW: empty file
│   └── notes.py        ← NEW: the /api/notes routes, moved from main.py
├── utils/
│   ├── __init__.py     ← NEW: empty file
│   └── for_testing.py  ← NEW: two small functions to practise testing on
└── tests/
    ├── test_reverse.py ← NEW
    └── test_average.py ← NEW
```

Run uvicorn from the same folder as before (`uvicorn main:app --reload --port 3001`).

What `__init__.py` is for. A folder of Python files you import from is a **package**. An empty `__init__.py` marks a folder as a *regular* package. Python can import from a folder without one

(a "namespace package"), but then a package with the same name installed in your venv would win. `utils` is a common name, so the empty file makes sure `from utils...` finds *your* folder.

Routers: routers/notes.py

An **APIRouter** is a group of routes that you later attach to the app. It works like `app`: you write `@router.get(...)` instead of `@app.get(...)`.

New file `routers/notes.py`. **Move** all five note routes here from `main.py`. Only two things change in each:

1. `@app.` becomes `@router.`.
2. The path drops the `/api/notes` start, because the router adds it (`prefix`): `"/api/notes"` becomes `""`, and `"/api/notes/{note_id}"` becomes `"/{note_id}"`.

The function bodies stay exactly the same.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from database import get_db
from models import Note
from schemas import NoteCreate, NoteOut, NoteUpdate

router = APIRouter(prefix="/api/notes")


@router.get("")
def get_notes(db: Session = Depends(get_db)) -> list[NoteOut]:
    notes = db.scalars(select(Note)).all()
    return [NoteOut.model_validate(n) for n in notes]


@router.get("/{note_id}")
def get_note(note_id: int, db: Session = Depends(get_db)) -> NoteOut:
    ...  # body unchanged


@router.delete("/{note_id}", status_code=204)
def delete_note(note_id: int, db: Session = Depends(get_db)) -> None:
    ...  # body unchanged


@router.post("", status_code=201)
def create_note(data: NoteCreate, db: Session = Depends(get_db)) -> NoteOut:
    ...  # body unchanged


@router.patch("/{note_id}")
def update_note(note_id: int, data: NoteUpdate, db: Session = Depends(get_db)) ->
    NoteOut:
    ...  # body unchanged
```

`prefix="/api/notes"` is put in front of every path in this router: `@router.get("/{note_id}")` answers `/api/notes/{note_id}`. The URLs don't change, so the frontend needs no changes. The

root route `/` stays in `main.py`: its path doesn't fit under the prefix, and it belongs to the app as a whole.

Logging: middleware.py

Until now you logged with `print`. Python has a built-in **logging** module, and it's the standard way for servers to log:

- **Levels.** Each message has a level: `DEBUG`, `INFO`, `WARNING` or `ERROR`. You choose in one place which levels are shown.
- **One place for the format.** Every message gets the same layout, set once.

How logging works. Three kinds of objects work together:

- A **logger** is what your code writes to. `logging.getLogger(__name__)` gives a file its own logger, named after the module (`__name__` is the module's name, here `"middleware"`). Calling `logger.info("...")` creates a **record**: the message, its level, the logger's name, the time.
- Loggers form a tree by name, and every logger passes its records up to the **root logger** at the top. So all the loggers in your app end up in one place.
- A **handler**, attached to a logger, decides where records go: the terminal, a file, a logging service. Each handler has a **formatter** that turns a record into a line of text.

A fresh program has no handlers at all, and `INFO` messages are simply dropped. Python only has an emergency fallback that prints `WARNING` and above. One call to `logging.basicConfig` (in `main.py`, below) sets the level and attaches one handler, which writes to the terminal, to the root logger. Because every logger passes its records up to the root, that one line configures logging for the whole app, including the logs of libraries you use. To send logs to a file later, you change the handler in one place, not every `print`.

New file `middleware.py`. **Move** `log_requests` and `database_unavailable` here from `main.py`, with three changes: the `@app.middleware(...)` and `@app.exception_handler(...)` lines are removed (there's no `app` in this file, so `main.py` registers the functions); `print` becomes `logger.info`/`logger.error`; and `request` in `database_unavailable` gets the type `Request`.

```
import logging
import time

from fastapi import Request
from fastapi.responses import JSONResponse

logger = logging.getLogger(__name__)


async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    logger.info(f"{request.method} {request.url.path} {response.status_code} {ms:.1f} ms
    ")
    return response


def database_unavailable(request: Request, exc):
    logger.error(f"database error: {exc.orig}")
```

```
    return JSONResponse(status_code=503, content={"detail": "database unavailable"})
```

main.py: wiring it together

Replace the whole `main.py` with:

```
import logging

from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from sqlalchemy.exc import OperationalError

from database import Base, engine
from middleware import database_unavailable, log_requests
from routers import notes

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

Base.metadata.create_all(engine)

app = FastAPI()

app.middleware("http")(log_requests)
app.add_exception_handler(OperationalError, database_unavailable)
app.include_router(notes.router)


@app.get("/", response_class=HTMLResponse)
def root() -> str:
    return "<h1>Hello World!</h1>"
```

Line by line:

- `logging.basicConfig(...)` configures logging once, for the whole app. Its `level` argument, `logging.INFO`, shows `INFO` and above, and `DEBUG` is hidden. `format` sets the layout, with placeholders that `logging` fills in for each message: the level name and the message text.
- The `app.middleware(...)` line registers the middleware. This is exactly what the decorator did before, because a decorator is just a function call (1b). These two do the same:

  ```
  @app.middleware("http")
  async def log_requests(request, call_next): ...

  async def log_requests(request, call_next): ...
  app.middleware("http")(log_requests)
  ```

  Without `@`, the function can live in another file.
- `app.add_exception_handler(...)` does the same for the error handler. It is the non-decorator form of `@app.exception_handler(OperationalError)`.
- `app.include_router(notes.router)` attaches every route from the router to the app.
- `from routers import notes` imports the *module*, and `notes.router` picks the router from it. In 4c you'll add `routers/users.py`, which also has a variable named `router`. Importing

modules keeps the two apart: `notes.router` and `users.router`.

The server log now shows two lines per request:

```
INFO: GET /api/notes 200 4.1 ms
INFO:     127.0.0.1:53088 - "GET /api/notes HTTP/1.1" 200 OK
```

The first comes from your middleware, the second from uvicorn's own access log. You don't need an "unknown endpoint" handler, unlike the original course: FastAPI already answers unknown URLs with `404 {"detail": "Not Found"}`.

Find All References. When you move code, check that everything still uses the right import. In VS Code, right-click a name such as `get_db` and choose **Find All References** (Shift+Alt+F12). Pylance lists every file that uses it.

Testing: why and what

So far you tested by hand, by clicking in `/docs` and the frontend. Automated tests are code that checks your code. You run them with one command, as often as you like, and they tell you right away when a change broke something.

This section covers **unit tests**: tests of single functions, with no server or database. Section 4b tests the API itself.

pytest

pytest is the most widely used test tool for Python. It's a development tool like Ruff, so it goes in the repo root `requirements-dev.txt`:

```
ruff==0.17.1
pytest==9.1.1
```

Install it from the repo root: `pip install -r requirements-dev.txt`.

What pytest does when you run it

1. **Finds its settings.** It looks for a config file such as `pytest.ini`, starting in the current folder and moving up. The folder where it finds one is the **rootdir**.
2. **Collects tests.** It walks through the folders and picks up files named `test_*.py`; inside them, functions named `test_*`; and classes named `Test*`, with methods named `test_*`.
3. **Imports each test file**, like any Python module.
4. **Calls each test, one at a time.** A test **passes** if it finishes without raising an exception. Any exception means it **fails**. pytest catches it, records it, and moves on, so one failure doesn't stop the run.
5. **Prints a report** and exits with code `0` if everything passed, or `1` if something failed. Tools that run tests automatically (GitHub Actions, in chapter 11) read that code.

There's no special "pass" or "fail" function. **Raising an exception is how a test fails.**

pytest.ini: where imports are looked up

New file `pytest.ini` in `notes_backend`:

```
[pytest]
pythonpath = .
```

To see why it's needed: when Python runs `from utils.for_testing import reverse`, it searches a list of folders called `sys.path`, in order, for a `utils` folder or a `utils.py` file. If none of them has it, it raises `ModuleNotFoundError`.

- **uvicorn** adds the folder you run it from to `sys.path`. That's why `from database import ...` has always worked in `main.py`.
- **pytest** adds the test file's own folder, here `tests/`. `utils/` isn't inside `tests/`, so without `pytest.ini` every test file fails with `ModuleNotFoundError: No module named 'utils'`.
- `pythonpath = .` adds the rootdir, the folder that holds `pytest.ini`, to `sys.path`. Now `utils` is found.

The file does a second job: it marks `notes_backend` as the rootdir, so pytest knows where the project starts.

Something to test: utils/for_testing.py

```
def reverse(text: str) -> str:
    return text[::-1]


def average(numbers: list[float]) -> float:
    return sum(numbers) / len(numbers)
```

`text[::-1]` is **slicing**, `[start:stop:step]`. Leaving out start and stop means "the whole string", and a step of `-1` walks it from the last character to the first. The result is a new, reversed string.

The first tests

New file `tests/test_reverse.py`:

```
from utils.for_testing import reverse


def test_reverse_of_a():
    assert reverse("a") == "a"


def test_reverse_of_react():
    assert reverse("react") == "tcaer"


def test_reverse_of_saippuakauppias():
    assert reverse("saippuakauppias") == "saippuakauppias"
```

How assert works. `assert <condition>` is a normal Python statement, not something added by pytest. If the condition is true, nothing happens. If it's false, Python raises `AssertionError`, and that exception is what makes the test fail.

On its own, an `AssertionError` doesn't say which values were compared. pytest fixes that: when it imports a test file, it **rewrites every assert** into code that remembers both sides of the comparison. When a test fails, the report shows what came out and what was expected. That's why pytest needs no special functions like `assertEqual`.

Give each test a name that says what it checks: the name is what you see when it fails.

New file `tests/test_average.py`:

```
from utils.for_testing import average


class TestAverage:
    def test_of_one_value_is_the_value_itself(self):
        assert average([1]) == 1

    def test_of_many_is_calculated_right(self):
        assert average([1, 2, 3, 4, 5, 6]) == 3.5

    def test_of_empty_list_is_zero(self):
        assert average([]) == 0
```

How a test class works. `class TestAverage:` groups related tests, like `describe` in the original course. It's an ordinary class used only as a container:

- For *each* test method, pytest creates a *new* object of the class and calls the method on it. That's why each method takes `self`, even though you don't use it.
- Because every test gets a fresh object, tests in the same class can't affect each other.
- Don't give the class an `__init__` method; pytest skips classes that have one.

Running the tests

From `notes_backend`, run `pytest`:

```
tests/test_average.py ..F
tests/test_reverse.py ...

    def average(numbers: list[float]) -> float:
>       return sum(numbers) / len(numbers)
E    ZeroDivisionError: division by zero

FAILED tests/test_average.py::TestAverage::test_of_empty_list_is_zero - ZeroD...
1 failed, 5 passed
```

- Each `.` is a passed test, and `F` a failed one.
- The failure is a `ZeroDivisionError`, not an `AssertionError`: `average([])` crashed before `assert` could compare anything. It still counts as a failure, because any exception does.
- `>` marks the line where it happened, and `E` shows the error.

The test found a real bug. `len([])` is 0, and dividing by 0 raises an error. Fix `average`:

```python
def average(numbers: list[float]) -> float:
    if len(numbers) == 0:
        return 0
    return sum(numbers) / len(numbers)
```

Run `pytest` again: `6 passed`. Useful options:

```
pytest -v                       # print each test's full name
pytest tests/test_average.py    # run one file
pytest -k empty                 # only tests whose name contains "empty"
```

Comparing dicts and lists

The original course needs a special `deepStrictEqual` to compare objects, because JavaScript's `===` checks whether two objects are the *same object* in memory. Python has two different operators:

- `is` asks "are these the same object?"
- `==` asks "do these hold equal values?" For a dict, it checks that both have the same keys and compares each pair of values with `==` again, all the way down.

```python
a = {"author": "Noor Haddad", "blogs": 3}
b = {"author": "Noor Haddad", "blogs": 3}
a == b    # True: same contents
a is b    # False: two separate objects

[{"likes": 1}] == [{"likes": 1}]   # True
```

The order of keys in a dict doesn't matter; the order of items in a list does. Plain `assert ... == ...` is all your tests need.

Python tools for the exercises

Each tool is shown on this list:

```python
blogs = [
    {"title": "A", "author": "Maren Holt", "likes": 7},
    {"title": "B", "author": "Idris Okafor", "likes": 5},
    {"title": "C", "author": "Idris Okafor", "likes": 12},
]
```

sum with a generator expression. `sum(b["likes"] for b in blogs)` gives 24. The generator expression doesn't build a list; it produces one value each time `sum` asks for the next. `sum` starts at 0 and adds each value as it arrives.

max with key.

```python
max(blogs, key=lambda b: b["likes"])   # {"title": "C", "author": "Idris Okafor", "likes
    ": 12}
```

`max` goes through the items one by one, calls the `key` function on each, and keeps the item whose key is the biggest so far. It only replaces it when it finds a strictly bigger one, so on a tie the

first item wins. It returns the *item itself*, not the key. `max([])` has nothing to return and raises `ValueError`, so check for an empty list first.

collections.Counter.

```
from collections import Counter

counts = Counter(b["author"] for b in blogs)   # Counter({"Idris Okafor": 2, "Maren Holt
    ": 1})

likes = Counter()
for b in blogs:
    likes[b["author"]] += b["likes"]           # Counter({"Idris Okafor": 17, "Maren
    Holt": 7})
```

A `Counter` *is* a dict (a subclass of `dict`) that maps each key to a count. Given a sequence, it adds 1 for each item it sees. With a normal dict, `likes["Maren Holt"] += 7` on a missing key raises `KeyError`, because += first reads the current value. A `Counter` returns `0` for a missing key instead, so += works on any key.

most_common.

```
counts.most_common(1)      # [("Idris Okafor", 2)]
Counter().most_common(1)   # []
author, count = counts.most_common(1)[0]
```

It sorts the keys from the biggest count down and returns the first `n` as a list of `(key, count)` tuples; on a tie, the key that was added first comes first. On an empty `Counter` the list is empty and `[0]` raises `IndexError`, so check for an empty list first.

Tabs and spaces: what Python actually checks

Sooner or later you will see `IndentationError: unindent does not match any outer indentation level` in a file that looks perfectly indented. The reason is usually a mix of tabs and spaces, which look identical in the editor but are different characters.

For each line, Python works out how deep the indentation is and compares it with the open blocks above. A tab counts as jumping to the next multiple of 8 columns, so one tab is column 8, while 4 spaces is column 4. If the open levels are column 0 (a class) and column 8 (its methods, indented with a tab), a line indented with 4 spaces matches neither, and Python can't tell which block it belongs to. Even when the columns happen to match, Python 3 refuses code where tabs and spaces could mean different depths.

`cat -A file.py` in a terminal shows a tab as `^I`. To fix a file, run `ruff format .` (which uses tabs because of `indent-style = "tab"`) or VS Code's **Convert Indentation to Tabs** command, and keep the VS Code settings from the start of the book so it doesn't happen again.

Exercises 4.1–4.7

These exercises build a new app: a **blog list**, where users save links to blog posts they find interesting. Each blog has a title, an author, a URL and a number of likes.

4.1: Blog list, step 1

Create `chapter4/bloglist_backend` with this single-file app as `main.py`:

```
import os

from dotenv import load_dotenv
from fastapi import Depends, FastAPI
from pydantic import BaseModel, ConfigDict
from sqlalchemy import create_engine, select
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column,
    sessionmaker

load_dotenv()

engine = create_engine(os.environ["DATABASE_URL"], pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine)


class Base(DeclarativeBase):
    pass


class Blog(Base):
    __tablename__ = "blogs"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str]
    author: Mapped[str]
    url: Mapped[str]
    likes: Mapped[int] = mapped_column(default=0)


class BlogCreate(BaseModel):
    title: str
    author: str
    url: str
    likes: int = 0


class BlogOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    title: str
    author: str
    url: str
    likes: int


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

```
Base.metadata.create_all(engine)

app = FastAPI()


@app.get("/api/blogs")
def get_blogs(db: Session = Depends(get_db)) -> list[BlogOut]:
    blogs = db.scalars(select(Blog)).all()
    return [BlogOut.model_validate(b) for b in blogs]


@app.post("/api/blogs", status_code=201)
def create_blog(data: BlogCreate, db: Session = Depends(get_db)) -> BlogOut:
    blog = Blog(title=data.title, author=data.author, url=data.url, likes=data.likes)
    db.add(blog)
    db.commit()
    db.refresh(blog)
    return BlogOut.model_validate(blog)
```

Make it a working project: copy `requirements.txt` from the notes backend, add a `.env` with the same `DATABASE_URL` (`create_all` creates a new `blogs` table), run it on a free port such as 3003, and check in `/docs` that blogs can be added and listed.

4.2: Blog list, step 2

Split the app into modules with the same structure as the notes example: `main.py`, `database.py`, `models.py`, `schemas.py`, `middleware.py` (a `log_requests` middleware that uses `logging`), and `routers/blogs.py` with an `APIRouter` whose prefix is `/api/blogs`. Move one piece at a time, and after each piece restart uvicorn and check `/docs`.

4.3: Helper functions and unit tests, step 1

Add `utils/__init__.py`, `tests/` and `pytest.ini`. In `utils/list_helper.py`, write `dummy(blogs)`, which takes a list of blogs and always returns 1. This test, in `tests/test_list_helper.py`, must pass:

```
from utils.list_helper import dummy


def test_dummy_returns_one():
    blogs = []
    assert dummy(blogs) == 1
```

In these exercises a blog is a plain dict with the same keys as the API's JSON. Type the parameter as `blogs: list[dict]`.

Test data for 4.4–4.7. Put this list in `tests/test_list_helper.py`:

```
blogs = [
    {"title": "Patterns for state", "author": "Maren Holt",
     "url": "https://example.com/state", "likes": 7},
    {"title": "Why GOTO hurts", "author": "Idris Okafor",
     "url": "https://example.com/goto", "likes": 5},
    {"title": "Shrinking strings", "author": "Idris Okafor",
     "url": "https://example.com/strings", "likes": 12},
    {"title": "First-class tests", "author": "Noor Haddad",
```

```
     "url": "https://example.com/tests", "likes": 10},
    {"title": "Does TDD hurt design?", "author": "Noor Haddad",
     "url": "https://example.com/tdd", "likes": 0},
    {"title": "Type wars", "author": "Noor Haddad",
     "url": "https://example.com/types", "likes": 2},
]
```

For each of the four functions below (`total_likes`, `favorite_blog`, `most_blogs`, `most_likes`), test three cases: an empty list, a list with one blog (`[blogs[0]]`), and the whole list. Put each function's tests in its own `Test...` class. Expected results:

Function	`[]`	`[blogs[0]]`	full list
`total_likes`	`0`	`7`	`36`
`favorite_blog`	`None`	`blogs[0]`	`blogs[2]` ("Shrinking strings")
`most_blogs`	`None`	`{"author": "Maren Holt", "blogs": 1}`	`{"author": "Noor Haddad", "blogs": 3}`
`most_likes`	`None`	`{"author": "Maren Holt", "likes": 7}`	`{"author": "Idris Okafor", "likes": 17}`

4.4: Helper functions and unit tests, step 2

`total_likes(blogs)` returns the sum of all likes.

4.5*: Helper functions and unit tests, step 3

`favorite_blog(blogs)` returns the blog with the most likes, as the whole dict. On a tie, any of them is fine. For an empty list, return `None`, and type the return value as `dict | None`.

4.6*: Helper functions and unit tests, step 4

`most_blogs(blogs)` returns the author with the most blogs and how many, as `{"author": ..., "blogs": ...}`. On a tie, any of them is fine; for an empty list, `None`.

4.7*: Helper functions and unit tests, step 5

`most_likes(blogs)` returns the author whose blogs have the most likes in total, and that total, as `{"author": ..., "likes": ...}`. On a tie, any of them is fine; for an empty list, `None`.

Limits for 4.3–4.7: tests use plain `assert`; `pytest` must pass, and `ruff check .` from the repo root must report `All checks passed!`.

b. Testing the backend

In 4a you tested single functions. Now you test the **API itself**: send real requests to your routes and check the responses and the database. These are **integration tests**, because they test several parts working together: FastAPI, your routes, SQLAlchemy and PostgreSQL.

We work in `chapter3/notes_backend`. The exercises then do the same for the blog list.

A separate database for tests

Tests delete and create data on every run. Running them against Neon would wipe your real notes. So tests get **their own database**. It runs **locally, in Docker**: fast, because there's no network trip to Neon, and its data doesn't matter.

How Docker works

- An **image** is a ready-made, read-only package: a program plus everything it needs to run. `postgres:17` is the official PostgreSQL 17 image; `17` is the **tag**, the version. We use 17 because Neon runs PostgreSQL 17, so tests run on the same version as production.
- A **container** is a running copy of an image. It's isolated from your computer: it has its own files, its own processes and its own network address. That's why PostgreSQL doesn't need to be installed on your system.

Starting the test database

Run this once:

```
docker run -d --name fso-test-db -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=fso_test -p
↪  5432:5432 postgres:17
```

Part	Meaning
`docker run ... postgres:17`	create a container from the image and start it; the image is downloaded the first time
`-d`	"detached": run in the background and give you the terminal back
`--name fso-test-db`	a name for later commands
`-e POSTGRES_PASSWORD=secret`	an environment variable for the container; the image reads it on first start and sets it as the password of the user `postgres`
`-e POSTGRES_DB=fso_test`	the image creates a database with this name on first start
`-p 5432:5432`	**port publishing**, `computer:container`. The container has its own network address that your programs can't reach directly. Docker listens on port 5432 of your computer and forwards each connection into the container's port 5432, where PostgreSQL listens.

Wait a few seconds after the first start. On first start the image creates the database, restarts PostgreSQL, and only then accepts connections. Connecting too early fails with "server closed the connection unexpectedly".

```
docker ps                   # list running containers
docker stop fso-test-db     # stop it (the data stays)
docker start fso-test-db    # start it again, for example after a reboot
docker rm -f fso-test-db    # delete the container and its data
```

The data lives inside the container, so it survives `stop` and `start` and is lost on `rm`. For a test database, that's fine.

VPNs and published ports

If every connection to a published port fails with "server closed the connection unexpectedly" (or `curl` reports "Connection reset by peer"), even after waiting, check your VPN. Some VPN apps block all traffic to private network addresses, and Docker's containers live on such addresses. In Mullvad, for example, the setting is **Local network sharing**; with it blocked, nothing on your machine can reach a container. Allowing it lets your computer reach private addresses (containers, and also devices on your home network) while internet traffic still goes through the VPN.

Connecting to it

Add a line to `chapter3/notes_backend/.env`:

```
DATABASE_URL=postgresql+psycopg://...your Neon URL, unchanged...
TEST_DATABASE_URL=postgresql+psycopg://postgres:secret@localhost:5432/fso_test
```

It has the same parts as the Neon URL from 3c: user `postgres`, password `secret`, host `localhost` (your computer, where Docker listens), port `5432` and database `fso_test`. The blog list will use the same test database; the two apps use different tables, so they don't affect each other.

Making the app use the test database: conftest.py

`database.py` creates the engine from `DATABASE_URL`. During tests, that variable has to hold the test URL *before* `database.py` runs.

New file `tests/conftest.py`:

```
import os

from dotenv import load_dotenv

load_dotenv()
os.environ["DATABASE_URL"] = os.environ["TEST_DATABASE_URL"]
```

How this works, step by step:

1. pytest imports `conftest.py` **before any test file in the same folder**. The name is fixed: pytest looks for exactly `conftest.py`. It's the place for setup that all test files share.

2. `load_dotenv()` loads `.env`, so both URLs are now in `os.environ`.

3. The next line copies the test URL into `DATABASE_URL`, replacing the Neon URL.

4. Then pytest imports your test file, which imports `main`, which imports `database`. `database.py` calls `load_dotenv()` again, but `load_dotenv` never replaces a variable that's already set (3c), so `DATABASE_URL` keeps the test URL. The engine connects to Docker.

It's also a safety net: if `TEST_DATABASE_URL` is missing, `os.environ[...]` raises `KeyError` and nothing runs, so tests can never fall back to Neon. `database.py` and `main.py` stay unchanged.

Calling the API in tests: TestClient

FastAPI's **TestClient** sends requests to your app from a test, with the same methods as httpx: `client.get(...)`, `client.post(..., json=...)`, and so on.

How it works without a server. Recall ASGI from 3a: your `app` is, underneath, one function that receives a request as Python data and sends the response back as Python data. uvicorn's job is to turn network bytes into that data. `TestClient` builds the same data directly and calls `app` itself: no uvicorn, no port, no network. That's why it's fast, and why you don't need to start the server before running tests. (The original course splits `app.js` from `index.js` so tests can import the app without starting a server. You already have that split: `main.py` only creates `app`, and uvicorn is started separately.)

Install its dependency. `TestClient` uses the **httpx2** package; without it, it falls back to the older `httpx` and prints a deprecation warning. Add it to `requirements-dev.txt`:

```
ruff==0.17.1
pytest==9.1.1
httpx2==2.15.1
```

Test helpers: tests/helper.py

```python
from sqlalchemy import select

from database import SessionLocal
from models import Note
from schemas import NoteOut

initial_notes = [
    {"content": "HTML is easy", "important": False},
    {"content": "Browser can execute only JavaScript", "important": True},
]


def notes_in_db() -> list[dict]:
    with SessionLocal() as db:
        notes = db.scalars(select(Note)).all()
        return [NoteOut.model_validate(n).model_dump() for n in notes]


def non_existing_id() -> int:
    with SessionLocal() as db:
        note = Note(content="willremovethissoon")
        db.add(note)
        db.commit()
        note_id = note.id
        db.delete(note)
        db.commit()
    return note_id
```

- `with SessionLocal() as db:` opens a session and closes it when the block ends, as in `cli.py`. Tests aren't routes, so there's no `Depends`.
- `notes_in_db` returns every note as a dict, in the same shape as the API's JSON: `model_validate` turns each row into a `NoteOut`, and `.model_dump()` turns that into a

plain dict. So you can compare it directly with `response.json()`.

What non_existing_id is for. It's for testing the **404 case**: asking for a note that doesn't exist. That test needs an id that is a real integer but matches no note. You could write `999999`, and it would almost always work, but it's a guess: if the table ever reaches that id, the test gets a real note back and fails for the wrong reason. `non_existing_id()` doesn't guess. It creates a note (PostgreSQL gives it the next id, say 57), deletes it, and returns 57. No note has that id now, and none ever will, because the sequence that generates ids never hands out the same number twice.

It's also different from asking for `/api/notes/abc`: `"abc"` isn't a number at all, so FastAPI rejects it with `422` before your route runs. To reach the `if note is None:` line in `get_note` and get the `404`, you need a valid integer that matches no note.

Resetting the database before each test: fixtures

Tests must not depend on each other or on leftover data. If one test adds a note, the next test's "there are 2 notes" check would fail. So before *each* test, the `notes` table is emptied and filled with `initial_notes`. pytest does this with a **fixture**: a function that pytest runs for you around a test.

New file `tests/test_notes_api.py`, the beginning:

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import delete

from database import SessionLocal
from main import app
from models import Note
from tests.helper import initial_notes, non_existing_id, notes_in_db

client = TestClient(app)


@pytest.fixture(autouse=True)
def reset_notes():
    with SessionLocal() as db:
        db.execute(delete(Note))
        db.add_all([Note(**n) for n in initial_notes])
        db.commit()
```

How fixtures work:

- `@pytest.fixture` registers the function as a fixture.
- Normally a test *asks* for a fixture by having a parameter with the fixture's name, `def test_x(reset_notes):`. pytest sees the name, runs the fixture first and passes its result in. This is the same idea as FastAPI's `Depends`: the function says what it needs, and the framework provides it.
- `autouse=True` means "run this before *every* test in this file, without being asked". Our tests don't use a value from it; they only need it to have run.
- By default a fixture runs once *per test*, which is exactly what resetting needs.
- A fixture can also `yield` a value: code before `yield` runs before the test, code after it runs after the test, the same pattern as `get_db`.

The new SQLAlchemy pieces:

- `delete(Note)` builds `DELETE FROM notes`, which removes every row. Like `select(...)`, it only builds the statement; `db.execute(...)` runs it.
- `db.add_all([...])` is `db.add` for a list. All rows are sent with one `commit`, in one transaction: one round trip instead of one per note. This is the Python version of the original course's "optimizing beforeEach".
- `Note(**n)` unpacks a dict into keyword arguments (1b).

Never hard-code ids in tests. The sequence keeps counting across runs, so the ids differ every time. Get them from `notes_in_db()`.

Why `from tests.helper import ...` `pythonpath = .` puts `notes_backend` on `sys.path`, so `tests.helper` means "the `helper` module inside the `tests` folder". This works whether or not `tests/` has an `__init__.py`.

The tests

```
class TestGetAllNotes:
    def test_notes_are_returned_as_json(self):
        response = client.get("/api/notes")
        assert response.status_code == 200
        assert response.headers["content-type"] == "application/json"

    def test_all_notes_are_returned(self):
        response = client.get("/api/notes")
        assert len(response.json()) == len(initial_notes)

    def test_a_specific_note_is_among_the_returned_notes(self):
        response = client.get("/api/notes")
        contents = [n["content"] for n in response.json()]
        assert "HTML is easy" in contents


class TestGetOneNote:
    def test_succeeds_with_a_valid_id(self):
        note_to_view = notes_in_db()[0]
        response = client.get(f"/api/notes/{note_to_view['id']}")
        assert response.status_code == 200
        assert response.json() == note_to_view

    def test_fails_with_404_if_note_does_not_exist(self):
        response = client.get(f"/api/notes/{non_existing_id()}")
        assert response.status_code == 404

    def test_fails_with_422_if_id_is_invalid(self):
        response = client.get("/api/notes/abc")
        assert response.status_code == 422


class TestAddNote:
    def test_succeeds_with_valid_data(self):
        new_note = {"content": "pytest makes testing easy", "important": True}
        response = client.post("/api/notes", json=new_note)
        assert response.status_code == 201
```

```
        notes_at_end = notes_in_db()
        assert len(notes_at_end) == len(initial_notes) + 1
        assert "pytest makes testing easy" in [n["content"] for n in notes_at_end]

    def test_fails_with_422_if_content_is_missing(self):
        response = client.post("/api/notes", json={"important": True})
        assert response.status_code == 422
        assert len(notes_in_db()) == len(initial_notes)


class TestDeleteNote:
    def test_succeeds_with_204_if_id_is_valid(self):
        note_to_delete = notes_in_db()[0]
        response = client.delete(f"/api/notes/{note_to_delete['id']}")
        assert response.status_code == 204

        notes_at_end = notes_in_db()
        assert len(notes_at_end) == len(initial_notes) - 1
        assert note_to_delete not in notes_at_end
```

What the response object gives you:

- `response.status_code` is the status, such as `200`;
- `response.headers["content-type"]` is a response header; `application/json` means the body is JSON;
- `response.json()` turns the body into Python dicts and lists.

What the tests check:

- **The database, not only the response.** A route could answer `201` and still not save anything. Only the database shows what really happened.
- **A rejected request changes nothing**: that's why the 422 test checks that the count is unchanged.
- **Invalid id: 422, not 400.** In the original course a malformed id reaches the code and must be turned into `400` by hand. Your route declares `note_id: int`, so FastAPI rejects `abc` before your function runs.
- `in` / `not in` check whether a list contains a value, comparing with `==`.

Running them

Make sure the container is running (`docker ps`), then run `pytest` from `notes_backend`: all tests pass.

Where did the log lines go? pytest **captures** everything printed or logged during each test and shows it only when that test fails, under "Captured log call". The original course has to switch off its logger in test mode; pytest makes that unnecessary.

Running only some tests:

```
pytest tests/test_notes_api.py                                    # one file
pytest tests/test_notes_api.py::TestAddNote                       # one class
pytest tests/test_notes_api.py::TestAddNote::test_succeeds_with_valid_data  # one test
```

```
pytest -k "add or delete"                                                    # names
↪  matching an expression
pytest -x                                                                    # stop at
↪  the first failure
```

The `file::Class::test` form is a **node id**; `pytest -v` prints each test's node id, so you can copy it from there.

Watch a new test fail once. A test that has never failed might check nothing, like an `assert total_likes(...)` without `==`, which passes for any non-zero number. After writing a test, break the code on purpose (for example change `status_code=201` to `200` in `create_note`), see the test fail, and undo the change.

Why the routes stay def, not async def

The original course rewrites its routes with `async`/`await` here. To see why this book doesn't, you need two words.

A **process** is a running program: uvicorn, with your app loaded, is one process. A **thread** is one line of execution inside a process: a sequence of instructions being carried out, one after another. A process starts with one thread and can start more; the threads share the process's memory (your module-level variables, the engine and its pool), and the operating system switches between them, so several can be in the middle of their work at the same time. When a thread waits, for example for the database to answer, it simply sits there, and the operating system runs other threads meanwhile.

FastAPI runs the two kinds of route differently:

- **async def routes** run on the **event loop** (2c): a single thread that switches between tasks whenever one reaches an `await`.
- **def routes** run in a **thread pool**: a set of threads started in advance and reused (at most 40 at once by default). Each request gets a thread, and a slow database call blocks only that thread.

Your database code is **synchronous**: `db.scalars(...)` simply waits, and there's nothing to `await`. In an `async def` route that wait would block the event loop, and *every* other request would stall until it finished. So with synchronous SQLAlchemy, `def` is the correct choice. (SQLAlchemy has an async version too, but it adds complexity this course doesn't need.) The original's section on forwarding errors to an error handler also needs nothing here: `HTTPException`, validation and your `OperationalError` handler already do that job.

Summary of changes in the notes example

File	Change
`.env`	add `TEST_DATABASE_URL`
`tests/conftest.py`	new: points `DATABASE_URL` at the test database
`tests/helper.py`	new: `initial_notes`, `notes_in_db`, `non_existing_id`
`tests/test_notes_api.py`	new: `reset_notes` fixture and API tests
`requirements-dev.txt`	add `httpx==2.15.1`
Docker	the `fso-test-db` container

Exercises 4.8–4.14

Setup for the blog list, in `chapter4/bloglist_backend`: `TEST_DATABASE_URL` in `.env` (same value as the notes backend), `tests/conftest.py`, `tests/helper.py` with `initial_blogs`, `blogs_in_db()` and `non_existing_id()`, and `tests/test_blogs_api.py` with a reset fixture.

Limits: tests use only the test database; no hard-coded ids; every test passes on its own and in any order; see each new test fail once.

4.8: Blog list tests, step 1

Test that `GET /api/blogs` returns the right number of blogs, as JSON. Run only the tests you're working on, using a node id or `-k`.

4.9: Blog list tests, step 2

Test that every returned blog has a key named `id` whose value is an `int`. In the original course the database names it `_id`; here `BlogOut` already uses `id`, so the test passes right away. To see it fail, temporarily rename `id` in `BlogOut`.

4.10: Blog list tests, step 3

Test that `POST /api/blogs` creates a blog: the status is `201`, the number of blogs in the database grows by one, and the new title is among the saved titles.

4.11*: Blog list tests, step 4

Test that a blog posted without `likes` is saved with `likes` equal to `0`, in the response and in `blogs_in_db()`. `BlogCreate` already has `likes: int = 0`, so it passes right away. To see it fail, temporarily remove the `= 0`. What status does the test get then, and why?

4.12*: Blog list tests, step 5

Test that a blog without `title`, and one without `url`, are both rejected with `422`, and that nothing is saved. (The original course expects `400`; FastAPI uses `422` for invalid request bodies, and that's correct here.)

4.13: Blog list expansions, step 1

Add `DELETE /api/blogs/{blog_id}`, answering `204`. Test that the blog is gone from the database and that the count drops by one.

4.14: Blog list expansions, step 2

Add `PATCH /api/blogs/{blog_id}` for updating a blog's likes, like the phonebook's number update in 3.17: a new schema `BlogUpdate` with one required field, `likes: int`; a missing id answers `404`; the response is the updated blog. Test a successful update (in the response and in the database), a missing id (`404`), and a body without `likes` (`422`).